TenderCheck Project Report

Contents

1 Business Rationale and Market Research

2 System Design and Models

3 System Implementation

4 Experiments

5 Discussion and Conclusion

1 Business Rationale and Market Research

1.1 Research Scope and Business Problem

This report examines how contractors in China check engineering bid documents before submission. In government procurement, clarifications and changes to tender documents are announced through correction notices [1]. Bid teams track the latest files and check evaluation criteria, prices, and submission materials. Each key requirement should have an accurate response that can be traced to the source document.

The review covers the main tender document, appendices, clarifications, and addenda. Requirements are spread across files, while responses appear in technical, commercial, and pricing materials. Tender coordinators, engineers, and commercial staff review the documents together, followed by a final check by the team lead. For example, if an addendum changes the completion period to 360 days while the draft bid says 380 days, the team locates both passages, confirms the latest requirement, and updates the draft.

1.2 Engineering Project Activity and Market Opportunity

The Chinese Ministry of Finance reported government procurement of engineering projects worth CNY 1.45 trillion in 2023, 1.38 trillion in 2024, and 1.30 trillion in 2025 [2][3][4]. These figures show substantial public procurement activity in engineering projects and a practical setting for TenderCheck.

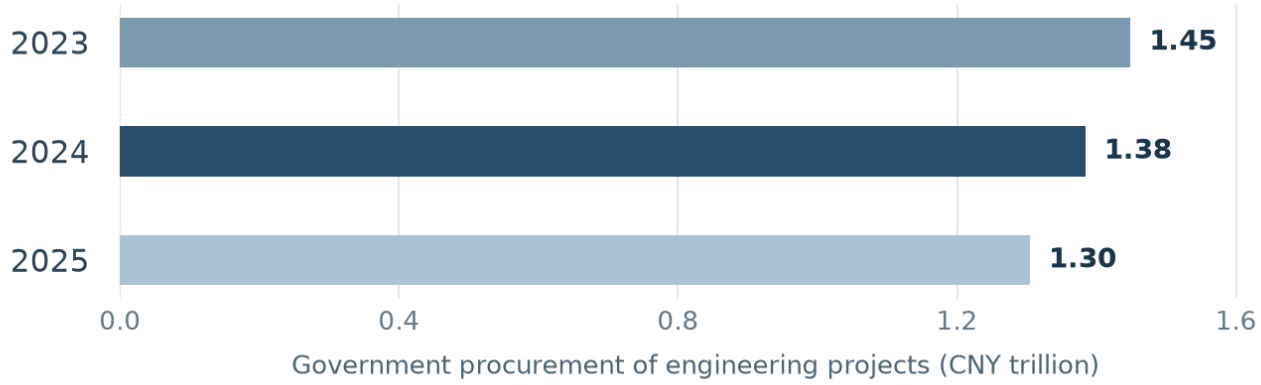


Figure 1 Government procurement of engineering projects in China (CNY trillion). The 2023-2025 figures are reported annual values, rounded to two decimal places. Sources: [2][3][4].

The Chinese government's procurement website and related platforms publish notices, clarifications, amendments, and results [1]. Bid teams track document versions and check the latest requirements against their draft before submission.

1.3 Existing Approaches and Product Entry Point

Bid teams often use manual checklists and PDF search. These methods are easy to adopt, while cross-file and version checks depend heavily on staff experience. For comparison, engineering document tools such as Provision and Document Crunch offer risk identification, specification review, and source citations [5][6]. The table summarizes these approaches and TenderCheck's current technical workflow.

Approach	Main use	Implications for TenderCheck
Manual checklists and PDF search	Read item by item, search keywords, and review as a team	Reflects the team's existing manual review workflow.

Approach	Main use	Implications for TenderCheck
General document Q&A	Locate and summarize parts of the documents quickly	The original text, version details, and page numbers help reviewers verify results.
Specialized engineering document tools	Risk review, specification extraction, and cross-document comparison [5][6]	Shows capabilities already available in engineering document review.
TenderCheck current prototype	Tender requirements → bid evidence → field extraction → rule checks	Workflow examined with S604 project cases.

Table 1 Comparison of pre-submission document review approaches.

1.4 Current Project Results and Practical Value

TenderCheck has developed modules for tender requirement annotation, classification, bid evidence retrieval, structured field extraction, and rule-based checks. The team used the S604 project to examine the workflow from tender requirements to bid evidence and rule outputs. S604 was set aside as a test project before model training. Retrieval results retain source files and page numbers, supporting item-by-item review against the original documents.

The modules address core work before bid submission: organizing key requirements, locating related passages in the bid, and checking explicit conditions such as completion periods, prices, and qualifications. Reviewers can use the retrieved evidence and rule prompts alongside the original text.

1.5 References

[1] Ministry of Finance, government procurement information notice.

[2] Ministry of Finance, 2023 procurement summary.

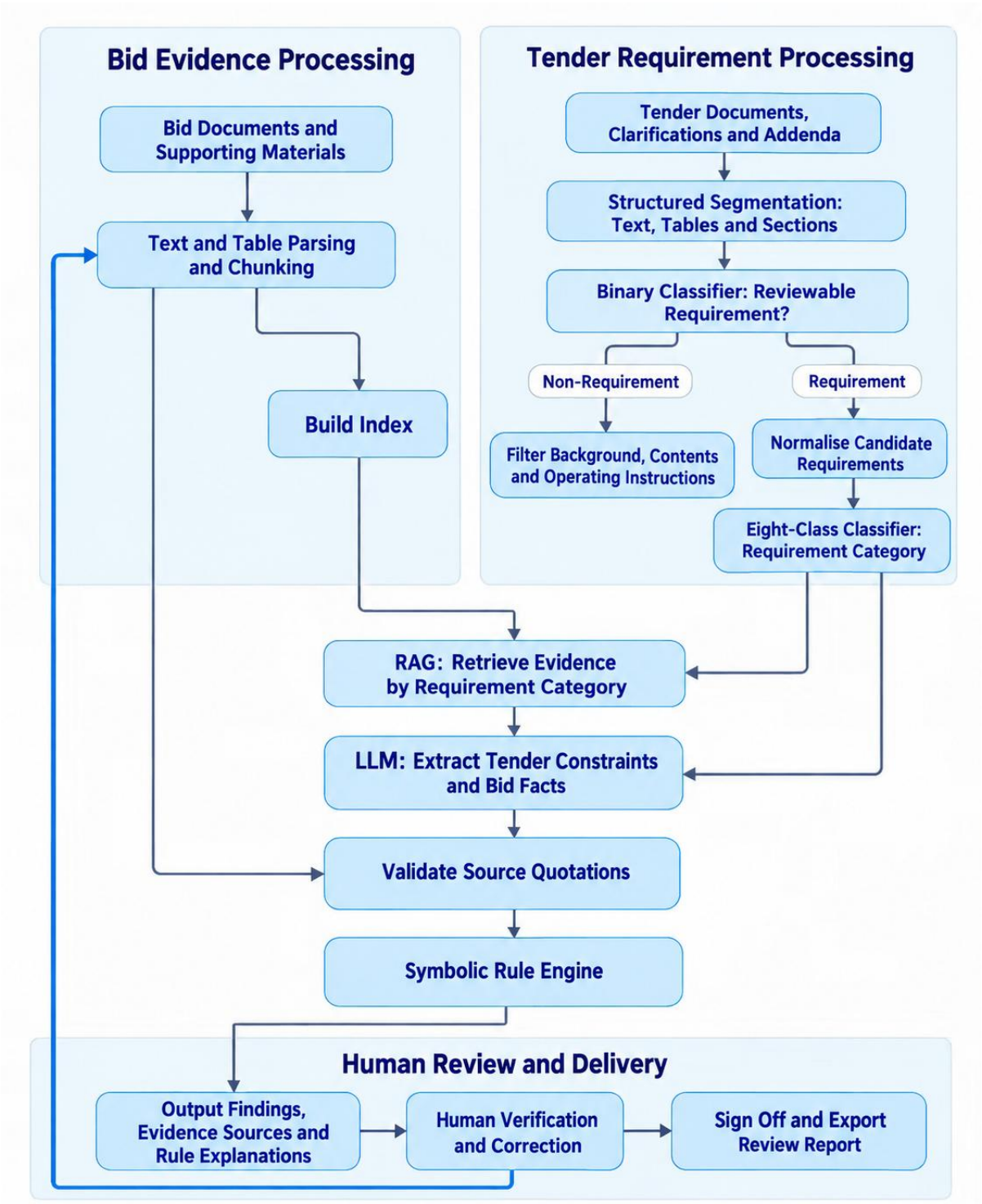
[3] Ministry of Finance, 2024 procurement summary.

[4] Ministry of Finance, 2025 procurement summary.

[5] Provision, product overview, 2026.

[6] Document Crunch, product guide, 2026.

2 System Design and Models



TenderCheck system design workflow from document intake to human review and report export.

2.1 Overall Architecture

TenderCheck is designed for pre-submission review of engineering bid documents. Its architecture combines machine learning, retrieval-augmented generation, large language models, and symbolic rules. The system accepts tender documents, clarifications and addenda, bid documents, and supporting materials. It outputs review conclusions, evidence sources, and rule explanations, with human review completing confirmation and report delivery.

The overall workflow has two input paths. The tender-requirement path identifies and classifies requirements to be checked; the bid-evidence path parses materials and builds an index. The paths meet at evidence retrieval, followed by information extraction, source-quotation validation, and rule evaluation. Each stage retains requirement identifiers, document sources, and page numbers so that results can be traced to specific clauses and original materials.

2.2 Tender Requirement Processing

Tender documents are first segmented structurally, converting body text, tables, and sections into candidate passages. Body-text processing identifies boundaries using clause numbers and section headings; table processing preserves the relationship between cells and row and column headings. Candidates retain the original text, document name, page number, and section information for classification and human verification.

A binary classifier decides whether a candidate passage is a requirement that must be checked at the current bidding stage. It filters background descriptions, contents pages, and some procedural instructions. The model combines text with section-structure features and uses a recall-oriented strategy for important sections to reduce omissions of key requirements.

Retained candidates undergo light normalization of table-header prefixes, check marks, and fill-in forms. A Chinese RoBERTa model then assigns one of eight classes: qualification and credentials, document composition and format, technical, commercial, time and schedule, consistency, integrity and compliance baseline, or scoring. The category guides later evidence retrieval and does not directly determine compliance. Complex clauses with several conditions may still need to be split further.

2.3 Bid Evidence Processing and Retrieval

The bid-evidence module reads bid documents and supporting materials, extracts body text, tables, and section information, and converts them into searchable text chunks. OCR may supplement scanned pages with insufficient extractable text; recognized content still needs to be checked against the original page.

Evidence is organized into parent and child chunks. Smaller child chunks match specific requirements, while more complete parent chunks provide context and reduce the loss of relationships caused by segmentation. Each chunk retains a document identifier, page number, section, text, and parent-child relationship so retrieved results can be located in the original file.

Once indexed, the RAG module retrieves bid evidence using the requirement text and its category. For example, qualification requirements prioritize certificates and personnel materials; commercial requirements focus on bid letters, guarantees, and pricing documents; schedule requirements focus on construction periods and progress plans. Results are ranked and duplicates consolidated before relevant passages and source information are passed to the extraction module.

The current Chinese upload workflow uses category-guided BM25 retrieval. The repository also contains a hybrid retrieval implementation combining vector search, fusion, and reranking. The actual retrieval configuration depends on the execution entry point. When no material is retrieved, the state should remain insufficient evidence; the system must not immediately conclude that the bidder failed to submit the document.

2.4 Large Language Model Extraction and Source-Quotation Validation

The large language model receives the tender requirement, requirement category, and retrieved evidence, and converts unstructured text into two groups of structured information: tender constraints and bid facts. Tender constraints include target field, comparison operator, value, unit, and applicability conditions. Bid facts include actual values, material status, entity identity, and corresponding evidence citations. The current main extraction interface uses DeepSeek.

Source-quotation validation checks whether quotations supplied by the model exist in the input passages and maps evidence identifiers to documents and page numbers. For key fields, it also checks whether values and units are supported by the source, whether the comparison operator matches the tender requirement, and whether personnel or certificate facts belong to the correct entity. Unconfirmed information remains unknown and is passed to the next evaluation stage.

In the architecture diagram, the bid-parsing module connects directly to source-quotation validation because validation needs the original text and source information. The eight-class module connects to both RAG and the large language model because the requirement category guides evidence location and constrains extraction. Citation validation can check textual support but cannot fully replace semantic judgment or verification of an original document's authenticity.

2.5 Symbolic Rule Engine

The symbolic rule engine selects comparison logic from structured constraints. It covers numeric limits, qualification levels, amount and entity consistency, document completeness, prohibited acts and commitments, staffing, past performance, and formula calculations.

For example, when the tender limits the construction period to 360 days and the bid states 380 days, the model extracts values, units, and citations; the rule engine compares the numbers and explains why the schedule constraint is not met. Separating rule evaluation from information extraction makes it easier to identify where an error occurred.

Output states include compliant, non-compliant, insufficient evidence, conflicting evidence, unsupported rule, and scored result. Missing required fields or valid citations preserve uncertainty. Scoring is handled separately: a score of zero does not directly mean a mandatory requirement was violated. Each result also stores the rule identifier, comparison process, and evidence sources for human review.

2.6 Human Review and Delivery

The Streamlit front end displays requirements, bid responses, field comparisons, rule explanations, and evidence pages together. Reviewers can confirm a result, correct a field, add evidence, and record their handling notes. Machine verdicts and human review states are stored separately, distinguishing automatically identified issues from final human decisions.

The arrow from human review and correction back to text and table parsing and chunking represents the feedback loop in which revised material re-enters evidence processing. The current code provides page-text editing, local re-evaluation, and a new-version review entry point. A full file update also requires checking that parsing, indexes, and review results correspond to the current version; local re-evaluation must not be treated as complete reprocessing of the entire file.

Delivery includes human sign-off and export of the review matrix. The sign-off service checks the review round, review states, and addendum handling; the current report page mainly provides Excel export. Project files and caches are stored in the project directory, while SQLite stores documents, requirements, review rounds, results, and action records for later queries and tracing.

2.7 Models

TenderCheck configures different models for different tasks. A binary classifier screens review requirements; an eight-class model assigns requirement categories; a vision model assists with scanned documents; retrieval and reranking models locate evidence; and a large language model extracts structured constraints and facts. Requirement categories, evidence passages, and a unified field schema connect the models. A symbolic rule engine performs the final comparison.

2.7.1 Binary Tender-Requirement Classifier

The binary model distinguishes tender requirements that need review from non-review content. Its inputs are candidate passages segmented from tender PDFs together with section and structure information. Its outputs are the probability that a candidate is a review requirement and the resulting filter decision.

The model combines TF-IDF text features with section and clause-structure features. It supports probability-calibrated LinearSVC and logistic regression linear classifiers; the current training script uses logistic regression. Training examples are built by aligning raw PDF candidates with human annotations, and training, validation, and test sets are divided by source project to reduce leakage of material from the same project across sets.

At inference, the screening threshold is adjusted by section to favor recall in critical review sections and retain content that may need checking. The repository report gives an ROC-AUC of 0.979 and accuracy of 96.54% on a test set of 37,500 candidates. These are candidate-classification metrics, not the accuracy of the complete bid-review system. Binary-classification evaluation report.

2.7.2 Eight-Class Tender-Requirement Model

Eight-class classification uses a model fine-tuned from Chinese-RoBERTa-wwm-ext. It receives requirements retained and normalized by the binary stage and assigns one of eight classes: qualification and credentials, document composition and format, technical, commercial, time and schedule, consistency, integrity and compliance baseline, or scoring.

The model input includes both requirement text and section heading. Section context helps distinguish similar wording: a clause mentioning a project manager may concern personnel eligibility, a proof document, or a scoring condition. A classification head uses the encoded text to output probabilities for eight categories and retains the predicted category and confidence.

The model is fine-tuned under supervision using manually annotated tender requirements, with project-grouped data splits. The existing test report covers 5,756 requirements, with 98.21% accuracy and 97.28% Macro-F1. On 581 requirements from two independent held-out projects, accuracy is 96.73%. The category guides retrieval and extraction; it is not itself a compliance conclusion. Test-set results; held-out-project results.

2.7.3 Vision Document-Parsing Model

The vision-parsing module provides a Qwen2.5-VL-7B-Instruct backend for scanned certificates, performance contracts, financial statements, and other materials that are difficult to extract as text. The model reads page images and converts recognizable content into body text, Markdown tables, and related page metadata for later chunking and retrieval.

The project also provides a fallback combining RapidOCR with image-color and layout heuristics, and uses page-level caching to reduce repeated parsing. The vision model and fallback are different processing paths: the current Chinese upload workflow mainly uses local OCR fallback, so not every scanned page should be described as having been processed by Qwen.

Seal or signature detection is used as an auxiliary clue and cannot independently prove material authenticity or the validity of a seal or signature. Vision-parsing implementation.

2.7.4 Hybrid Evidence-Retrieval Model

The full hybrid retrieval configuration combines Okapi BM25 and BAAI/bge-m3 for lexical and semantic matching, respectively.

BM25 builds a sparse index from Chinese character segments, alphanumeric strings, and identifiers. It is useful for locating certificate numbers, highway numbers, amounts, and schedule expressions. BGE-M3 encodes requirements and evidence passages as dense vectors to recall similar meanings expressed with different words, such as linking 'similar engineering experience' to 'recently completed project record.'

The two result lists are merged by reciprocal rank fusion (RRF), using ranking information from both lexical and semantic matches. Requirement categories also adjust queries and document scope: qualification requirements focus on certificates and personnel materials, while commercial requirements focus on prices and bid guarantees.

The current new Chinese-project upload entry point uses category-guided BM25 retrieval; the full hybrid configuration is provided by a separate index implementation. A description of system models should distinguish these execution configurations. Hybrid-retrieval implementation.

2.7.5 Evidence-Reranking Model

Hybrid retrieval uses BAAI/bge-reranker-v2-m3 for reranking. Unlike vector retrieval, which encodes requirements and evidence separately, the reranker jointly inputs a requirement and each candidate passage to score their relevance more closely.

This step distinguishes material on the same topic from material that can support the current check. For example, two certificates may belong to the same category but only one may correspond to the proposed person. Reranking can improve candidate order; subsequent extraction and citation validation must still confirm the entity relationship.

The retrieval workflow returns the corresponding parent passage using parent-child chunk links. It normally outputs the top five pieces of evidence and retains document, page, and ranking information. Relevance scores are for ranking, not compliance probabilities. Retrieval and reranking entry point.

2.7.6 Structured Fact-Extraction Model

Structured extraction uses DeepSeek-Chat, with prompts and field contracts constraining the output. Inputs are the tender requirement, its category, and retrieved evidence; the output separates tender constraints from bid facts.

Tender constraints specify the target field, comparison operator, expected value, unit, and applicability conditions. Bid facts specify the actual response, material status, entity identity, and citation. For example, the model can convert 'the construction period must not exceed 360 days' into an upper-bound constraint and extract the actual period from the bid passage.

Extracted information follows a unified schema, with evidence identifiers and verbatim citations for key fields. The program then checks that citations exist and that values, units, and entity relationships are supported. Fields without support should remain unknown rather than be filled to match an expected conclusion.

This module primarily uses inference with a pretrained model, prompt constraints, and programmatic validation; the project does not retrain the base large language model. Validated structured information passes to the symbolic rule engine, keeping text understanding separate from rule evaluation. Extraction-model implementation.

3 System Implementation

TenderCheck connects project intake, document processing, automated verification, human review, remediation and report delivery. The implementation follows two input streams: tender documents define what must be checked, while bid documents provide the evidence. These streams meet in retrieval and structured extraction, followed by quotation validation and symbolic rules. Reviewers inspect the evidence, correct the bid response and export the resulting review record.

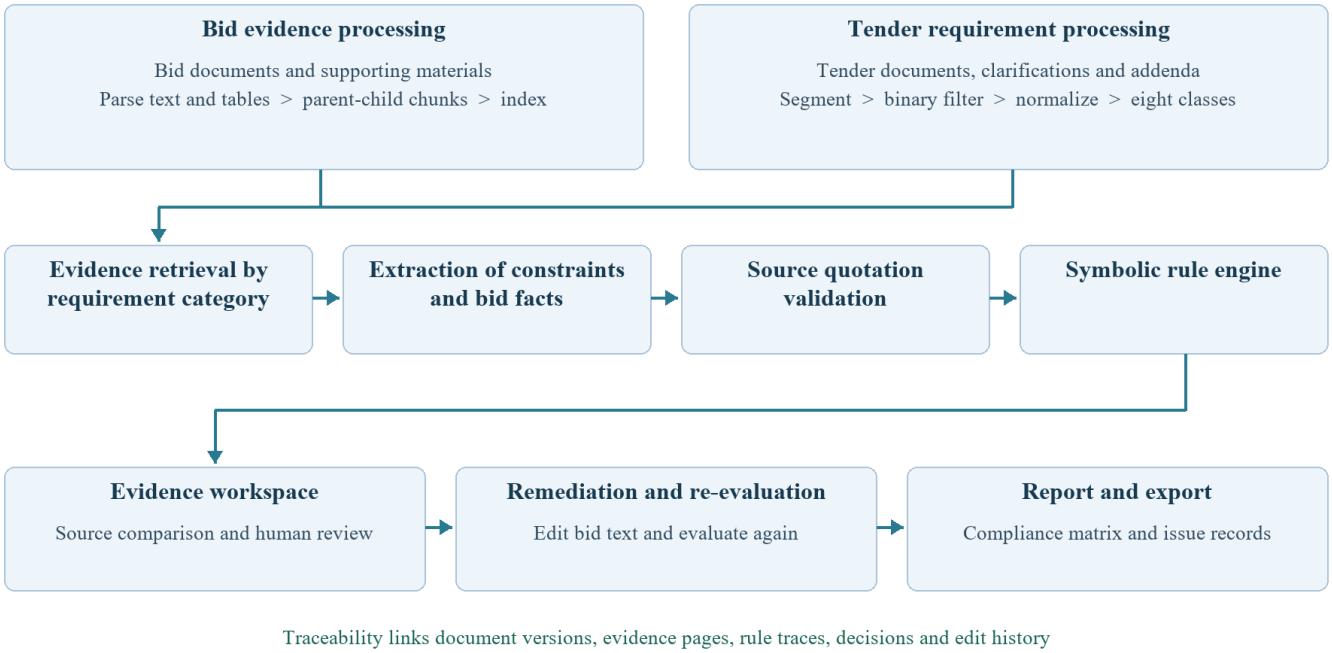


Figure 3.1. Implemented processing flow from documents to review delivery.

Implementation progress across the workflow

Stage	Implemented capability
1 Project intake	Create a named project and enter the document upload workflow.
2 Document processing	Confirm the file set, parse documents and submit the review pipeline.
3 Internal processing	Filter and classify requirements, retrieve evidence, extract facts and execute rules.
4 Evidence workspace	Compare source pages, inspect findings and record reviewer actions.
5 Remediation	Edit bid-page text, retain changes and re-evaluate the selected clause.
6 Report delivery	Filter the compliance matrix and export review and remediation records.

This chapter follows the current source and running interface. Screenshots establish visible functionality; their counts are not measures of accuracy. The interface and services share a Python application, with Streamlit, SQLite, local classification models and a language-model adapter providing the main implementation components.

3.1 Project Intake through New Project

New project is the entry point for a new review package. The form accepts a project name, rejects an empty name and creates a project record before switching to its workspace. The project identifier then links uploaded documents,

requirements, review rounds and reviewer actions. This gives each review a defined scope and prevents unrelated document sets from being treated as one case.

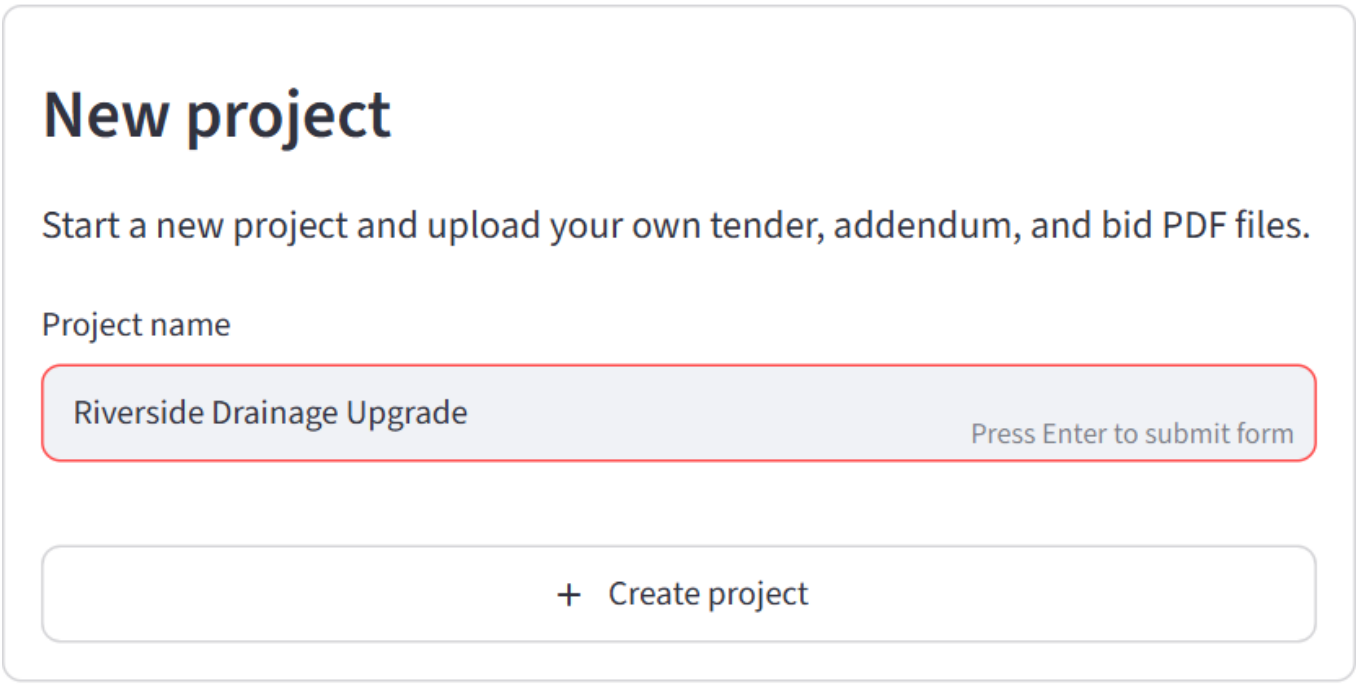


Figure 3.2. New project form for naming and creating a review project.

From project creation to document upload

After creation, the user uploads tender, addendum or clarification, and bid PDFs through the project upload panel in Overview. Creation and upload are consecutive actions: the New project form establishes the record, and the upload panel attaches the actual files. The service checks the PDF signature, detects identical files using SHA-256 and records version, category and parsing status.

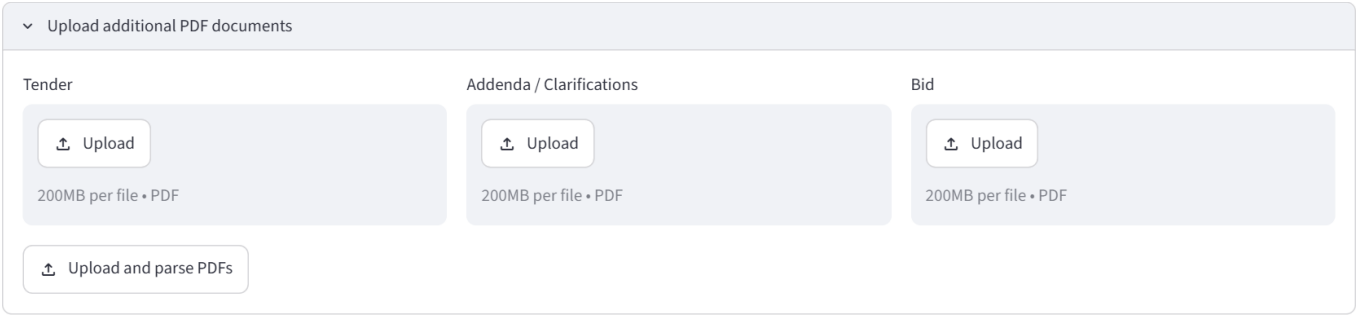


Figure 3.3. Document upload controls reached after project creation.

```
if category not in config.DOC_CATEGORIES:
    raise ServiceError(f"Unknown document category: {category}")
if not data.startswith(b"%PDF"):
    raise ServiceError(f"{filename} is not a PDF file.")
sha = hashlib.sha256(data).hexdigest()
existing = store.find_document_by_hash(project_id, sha)
if existing:
    return existing["id"], False
```

Figure 3.4. File-type validation and duplicate-file detection.

For bid-review teams, this intake sequence supports orderly handling of multiple volumes and revisions. For research, retaining document identity and version provides a basis for tracing an extracted requirement or a later evaluation result back to its input.

3.2 Tender and Bid Processing in Overview

Overview turns the uploaded files into an explicit review package. It separates tender and addenda, primary bid volumes and supporting attachments, and shows file counts, page counts, filenames and parsing status. Reviewers confirm that the intended documents are present before starting extraction and verification. This addresses a practical source of review errors: applying the right check to an incomplete or outdated file set.

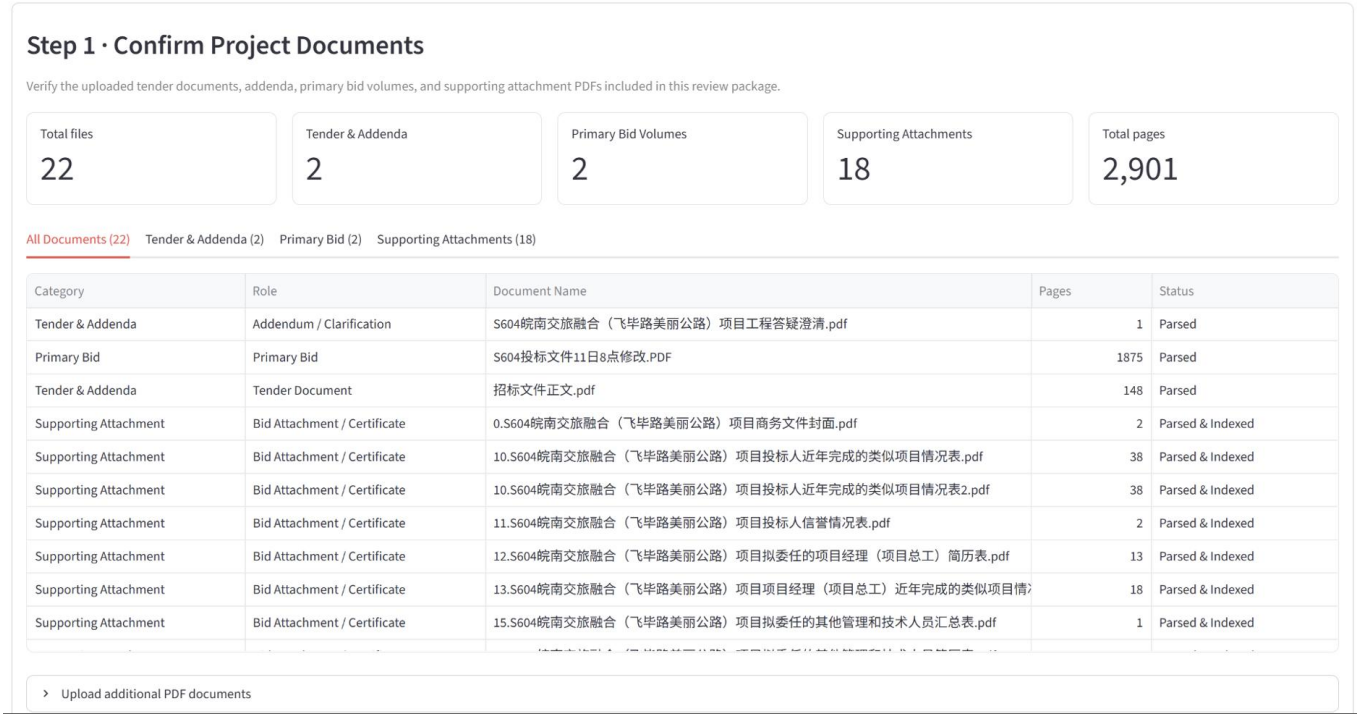


Figure 3.5. Overview document inventory and parsing status.

Submitting and tracking the processing pipeline

The pipeline action coordinates requirement extraction, evidence retrieval, structured fact extraction and rule evaluation. Service functions submit background jobs and store progress and outputs, allowing the interface to report the current stage. Completed results belong to a review round and can be opened in Review Workspace; later file changes can initiate another run.

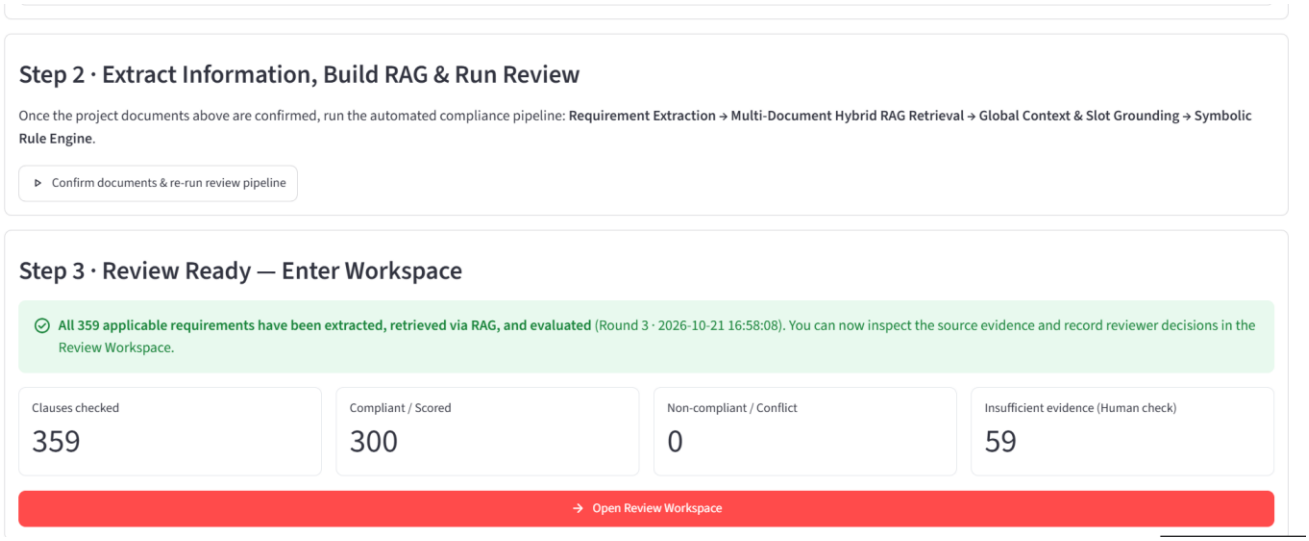


Figure 3.6. Overview action that starts or reruns the processing pipeline.

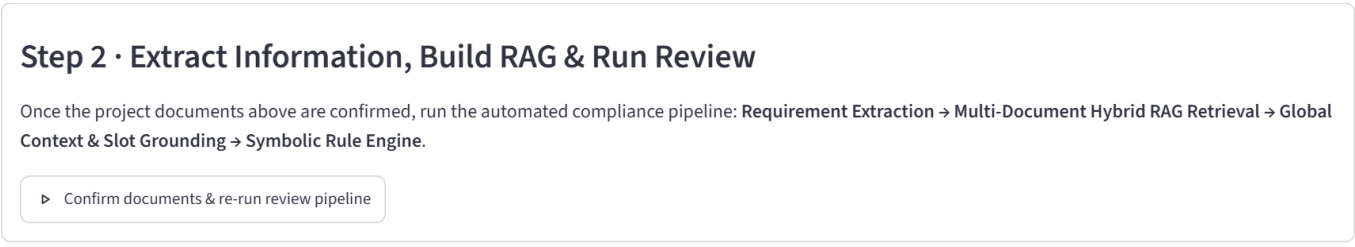
Processing responsibility	Output available to the next stage
Tender-side preparation	Clauses with source location, requirement text and category.
Bid-side preparation	Parsed pages, table representations and searchable evidence chunks.
Review coordination	Job state, review-round identifier and saved clause-level findings.

Keeping these intermediate outputs makes the workflow inspectable. Operationally, progress feedback and a clear review-round boundary help a team coordinate long-running work. Academically, the same boundaries allow errors in parsing, retrieval and reasoning to be examined separately.

3.3 Internal Processing and Automated Verification

Structural segmentation and binary requirement filtering

Tender documents and addenda are segmented using text, tables and section structure. The binary requirement model decides whether a candidate contains a review requirement. Chapter-aware filtering reduces background text, contents entries and instructions that should not become independent checks. A fallback retains high-scoring candidates if filtering produces no result, supporting recall without claiming that omissions are eliminated.




```
retained_pairs = self.binary_filter.filter_candidates(
    all_candidates,
    threshold=self.binary_threshold,
    chapter_aware=self.chapter_aware,
)
```

Figure 3.7. Binary filtering of candidate clauses with threshold and chapter context.

Normalization and the eight-class model

Retained candidates are normalized into requirement sentences. The eight-class model receives the requirement together with its section context and assigns a category. The local implementation loads the binary model and the roberta_wwm_ext classifier; each output retains source text, location and confidence information. The categories organize later evidence searches and reviewer filters rather than serving as compliance verdicts.

```
normalized_reqs: List[str] = []
cls_inputs: List[Dict[str, str]] = []
for cand, _ in retained_pairs:
    norm_req = normalize_candidate_to_requirement(cand)
    normalized_reqs.append(norm_req)
    cls_inputs.append(
        {
            "Section": cand.section or cand.chapter,
            "Requirement": norm_req,
        }
    )

probs_8 = self.classifier.predict_proba(cls_inputs)
```

Figure 3.8. Normalized requirement text and section context passed to the eight-class model.

Category	Review focus
Qualification	Licences, qualifications, personnel eligibility and past performance.
Document	Required documents, signatures, seals and submission format.
Technical	Technical standards, construction methods and quality requirements.
Commercial	Prices, guarantees and commercial commitments.
Schedule	Deadlines, validity periods and completion dates.
Consistency	Agreement of entities, amounts, versions and statements.
Integrity	Prohibited conduct and integrity commitments.
Scoring	Scoring conditions, weights and calculation rules.

This two-stage design separates requirement detection from topic classification, making each model independently evaluable. For users, the resulting checklist reduces manual sorting and directs attention to the evidence needed for each type of requirement.

Evidence retrieval and structured fact extraction

Bid text and tables are parsed into parent and child chunks. Page-level parents retain context, while shorter children support BM25 retrieval. The Chinese upload path uses category-aware search to return up to five parent passages. DeepSeekSlotExtractor then separates tender constraints from bid facts, and global-context completion adds relevant parameters from other document sections. If model extraction fails, the current adapter invokes an offline fallback.

```
local_passages = [dict(p) for p in passages_by_req_id.get(rid, [])]
ev_str = json.dumps(local_passages, ensure_ascii=False)
try:
    ext = deepseek_extractor.extract(
        cid,
        r_text,
        r_cat,
        ev_str,
        review_reference_date=ref_date,
    )
except Exception:
    ext = _offline_chinese_slot_fallback(cid, r_text, r_cat, ev_str, ref_date)

ext, hydrated_str = hydrate_with_global_context(ext, ev_str, global_ctx,
normalizer=deepseek_extractor)
final_passages = json.loads(hydrated_str) if hydrated_str else local_passages

rule_res = rule_engine.evaluate(ext)
out = format_backend_result(r_text, r_cat, ext, rule_res, final_passages, doc_sources)
outputs_by_req_id[rid] = out
```

Figure 3.9. Fact extraction, fallback, context completion and rule evaluation.

Retrieval-augmented extraction links the language model to inspectable evidence, addressing the research need for grounded reasoning. In practical review, page-level context helps distinguish a requirement from the bidder's actual response. Combined with global-context anchor completion, the pipeline avoids missing key parameters from the bid letter or summary schedules when a single local retrieval window is limited.

Quotation validation and symbolic rule execution

The validation layer checks evidence identifiers, exact source quotations and referenced fields. Unsupported values remain unknown; text alone does not establish that a signature or seal is present. The rule engine then evaluates supported facts using rules R01 to R09 and returns a verdict, rule identifier, explanation and evidence references. Numeric rules normalize units and use Decimal comparisons.

```
    passage = by_id.get(ref.get('evidence_id'))
    quote, field = ref.get('quote'), ref.get('field')
    if (passage is None or not isinstance(quote, str) or not compact(quote)
            or compact(quote) not in compact(passage['text']) or field not in
b.__dataclass_fields__):
        issues.append('Reference is not an exact quote from the supplied passage'); continue
    quote = original_quote_span(passage['text'],quote)
    verified.append(dict(field=field, evidence_id=ref['evidence_id'], quote=quote,
                         document_id=passage['document_id'], source_sha256=passage['source_sha256'],
                         page_start=passage['page_start'], page_end=passage['page_end']))
    by_field.setdefault(field, []).append(quote)
b.evidence_references = verified
b.evidence_pages = sorted({page for r in verified for page in range(r['page_start'],
r['page_end']+1)})
# Text alone cannot certify the actual image's stamp or signature.
b.has_official_seal = b.has_signature = None
```

Figure 3.10. Source-quotation validation and preservation of unknown signature or seal states.

```
expected = Decimal(str(t.expected_numeric)) * Decimal(str(eu[1]))
actual = Decimal(str(b.actual_numeric)) * Decimal(str(au[1]))
compare = {
    "<=": operator.le,
    "<": operator.lt,
    ">=": operator.ge,
    ">": operator.gt,
    "==": operator.eq,
}.get(t.operator)
```

Figure 3.11. Unit normalization and operator selection for numeric checks.

3.4 Evidence Inspection in Review Workspace

Review Workspace connects each requirement to its original tender page, relevant bid pages and the recorded result. Its clause list and filters support navigation by verdict or review status, reason tag, category and clause number. A reviewer can therefore move from a high-level issue list to the exact supporting document without manually locating each reference again.

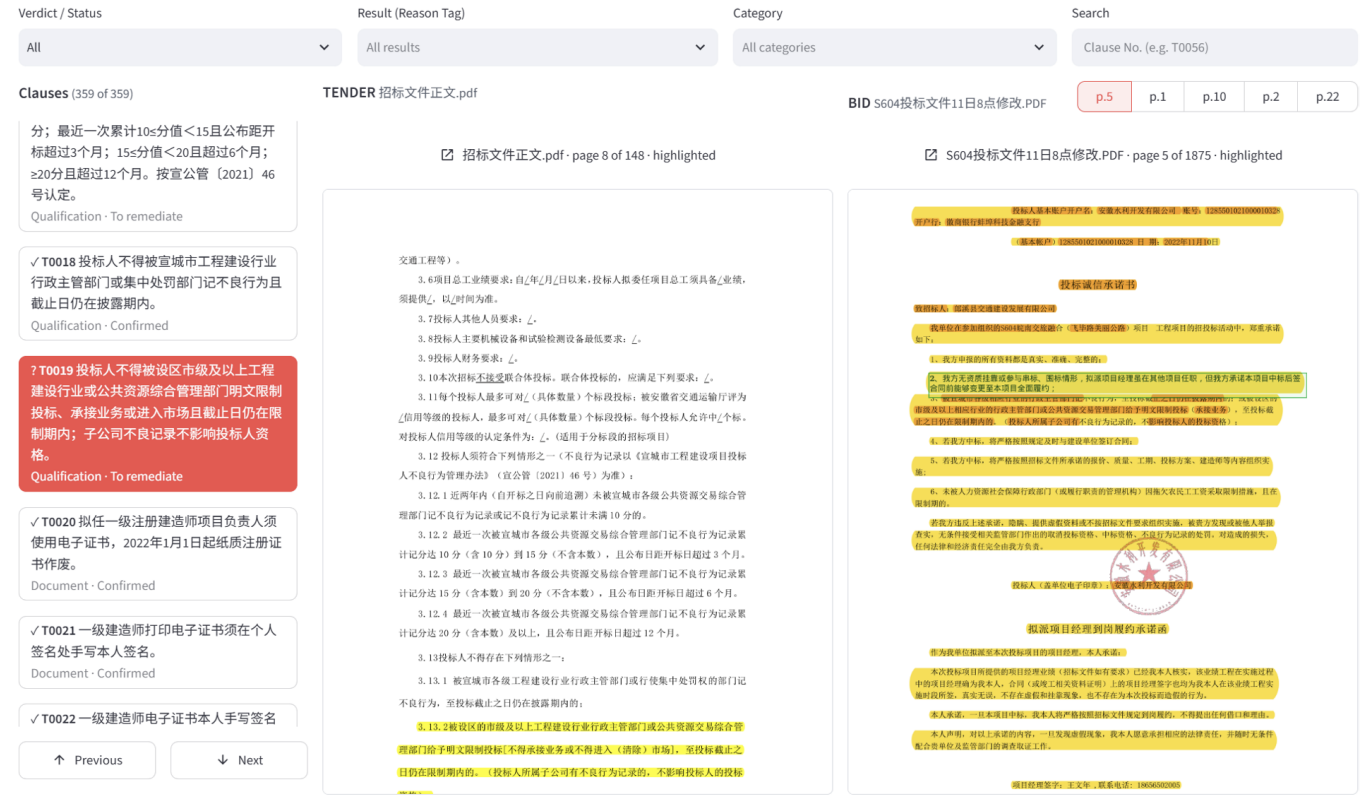


Figure 3.12. Clause navigation and side-by-side tender and bid evidence pages.

Workspace control	Implementation and user value
Verdict / Status filter	Filter clauses by primary system verdict (Compliant, Non-compliant, or Insufficient evidence) or human review status (Unreviewed or To remediate) while retaining the selected clause, helping reviewers prioritise unresolved items.
Result (reason tag) filter	Filter clauses by the nine fine-grained rule-engine reason tags: Compliant, Scored, Non-compliant, Conflicting Evidence, Seal and Signature Check, External Web Check, Performance Linkage, Subjective Scoring, and Attachment or Formula.
Category filter	Filter clauses by the eight requirement categories predicted by the classifier (Qualification, Document, Technical, Commercial, Schedule, Consistency, Integrity, and Scoring).

Workspace control	Implementation and user value
Search box	Locate a target clause directly by clause identifier (for example, T0019 or T0056) or requirement keyword.
Tender and addendum view	Switch between amended addendum clauses and original tender requirements to verify the effective source of the current constraint.
Bid evidence navigation (p.5, p.1, etc.)	Switch across matched bid evidence pages and highlight the exact supporting passage on the rendered PDF page.
Open source documents	Open the full-page document viewer to inspect surrounding page and cross-page table context beyond the extracted passage.

The arrangement supports evidence-based human decision making: a result can be challenged against its source. This is relevant to explainable AI and human-in-the-loop research, and to the market need to reduce repeated document navigation while keeping reviewers responsible for decisions.

Clause-level comparison and reviewer decisions

Below the source viewers, the clause panel presents the current requirement, selected bid content, extracted response field and rule result. It keeps the system verdict separate from human review status. A failed check can therefore be awaiting review, under remediation or already reviewed without silently changing the underlying machine result.



Figure 3.13. Requirement comparison, rule result and reviewer actions for a selected clause.

Inspection and action within one review context

Comparison field or control	Interface meaning and implemented behaviour
Evidence passage tabs (E1 to E5)	Switch among the top five retrieved bid passages and page numbers for the current clause; the primary verified passage is linked to both the extracted text panel and the right-hand PDF page viewer.

Comparison field or control	Interface meaning and implemented behaviour
Field	Displays the specific check dimension or structured constraint extracted from the tender clause (for example, substantive response and prohibition commitment in Figure 13, or a numeric schedule limit).
Bid response	Displays the bidder's actual response value, commitment statement, or qualification fact extracted from the selected bid passage and verified against the source quotation.
Result	Displays the symbolic rule-engine comparison outcome (Compliant, Non-compliant, or Insufficient evidence) together with the fine-grained diagnostic reason banner below the table (such as External Web Check when external administrative records must be verified).
Reasoning chain	Exposes the recorded classification, retrieval, slot extraction, and symbolic rule evaluation stages; retrieval scores describe passage relevance rather than compliance confidence.
Confirm	Records the reviewer's confirmation decision for the current clause and advances to the next unreviewed non-passing clause.
To remediation	Creates or links a remediation item and opens the Remediation editor with the selected clause and bid page in context.
History	Shows recorded timestamp, actor, action, and note for accountability and multi-round audit traceability.

The workflow converts an algorithm output into a reviewable action. Its academic relevance is the preservation of intermediate explanations and human feedback; its practical value is a shorter path from locating an issue to assigning a correction. Storing the machine verdict separately from the human review status ensures end-to-end auditability from initial automated screening to final human sign-off.

3.5 Bid Correction in Remediation

Remediation retains the selected requirement while opening the associated bid document and page. The left list distinguishes open and resolved items; the editor shows the current bid-page text. Reviewers can request an AI suggestion, select a document volume or page, edit the response and save it before returning to the workspace. This turns issue detection into an actionable correction process.

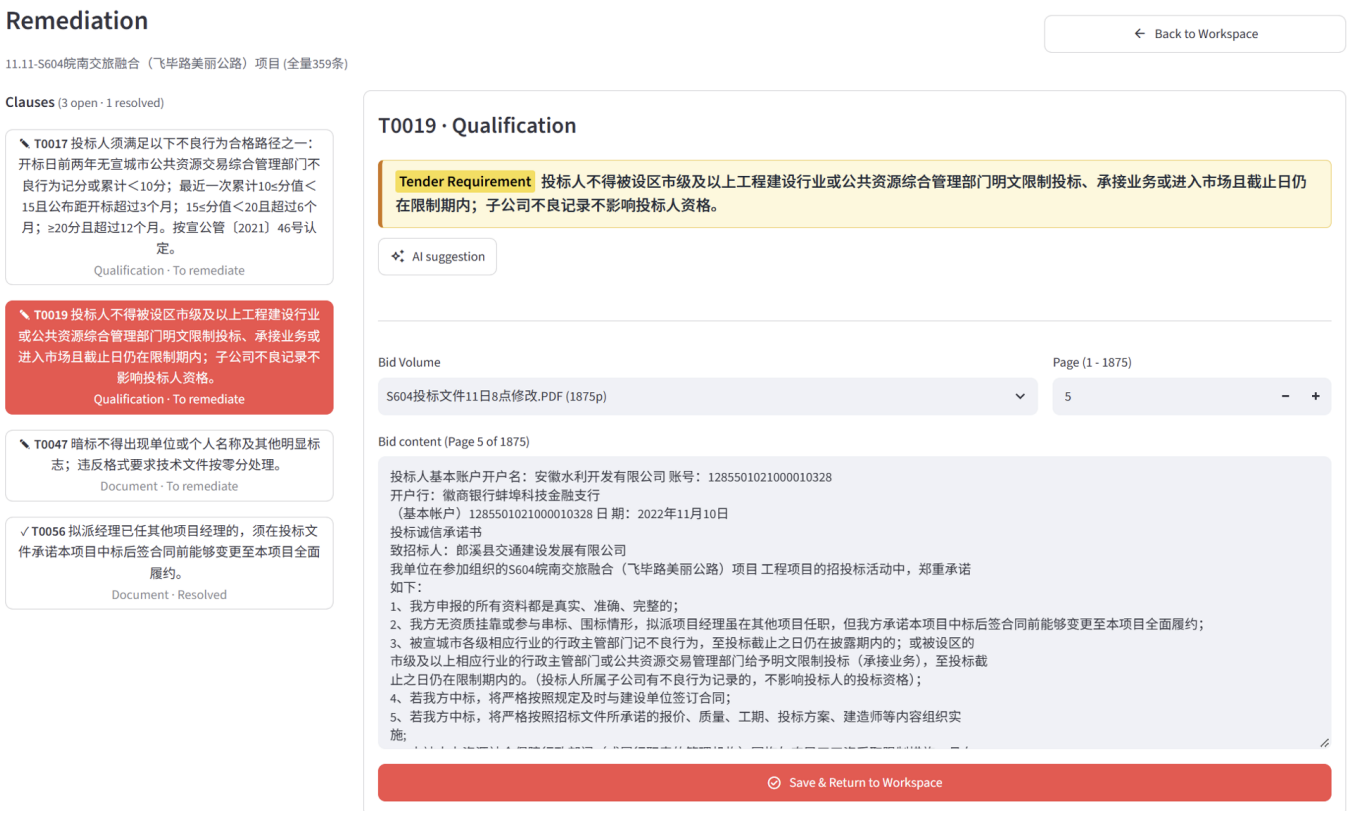


Figure 3.14. Remediation issue list, page selection, text editor and save action.

Saving an edit triggers re-evaluation

The service stores modified page text with its source path and page number, preserving the original text in a separate edit record. It then re-extracts the selected clause's facts and reruns the rule engine. An item closes only when the new verdict is compliant; otherwise it remains in remediation. The application uses a text-edit overlay, so saving here should not be described as rewriting the original PDF binary.

```
# Re-run real backend extraction & rule engine on the modified Bid text
reeval = real_backend.reevaluate_clause_with_modified_bid(
    req=req,
    existing_result=res,
    pdf_path=doc_path,
    page_no=int(page_no),
    modified_text=modified_text.strip(),
)

passed_now = reeval["verdict"] == COMPLIANT
rem_status = CLOSED if (resolve and passed_now) else TO_REMEDIATE
```

Figure 3.15. Re-evaluation and conditional closure after a bid-text edit.

This supports an auditable human-feedback loop: changes can be linked to revised findings rather than merely marked complete. It addresses the team's need to track issue resolution. AI suggestions remain proposed wording for human assessment; their generation depends on the configured model service.

3.6 Review Delivery through Report and Export

Report & Export aggregates the latest completed review round into a compliance matrix. Summary indicators show total clauses, compliant items, non-passing items and review or remediation progress. View filters focus on all

clauses, non-passing clauses or remediated clauses. The matrix links each clause to its category, tender requirement, bid response, bid evidence, system verdict and review status.

3.7 Technical Challenges and Implemented Responses

The main implementation challenges arise at the boundaries between document structure, statistical models, evidence and human decisions. The following responses are present in the current code; the remaining limits define what still needs engineering or evaluation before broader use.

Challenge	Implemented response	Remaining boundary
Long documents and table structure	Use section-aware segmentation, table representations and page-level parent chunks.	Scans, merged cells and cross-page tables can still lose information.
Requirement omissions and noisy categories	Separate binary filtering from eight-class classification; preserve text, confidence and fallback candidates.	Cross-domain transfer beyond highway engineering projects requires further evaluation.
Local retrieval misses global parameters	Add project-wide context and document anchors after category-aware retrieval.	Anchor relevance and completion errors require cross-project checks.
Unsupported facts and inconsistent units	Validate quotations, retain unknown values, normalize units and apply deterministic rules.	Literal matching is not full semantic validation; signatures and subjective scores need human assessment.
Repeated work and long-running tasks	Use a submission lock, active-job checks and saved job progress.	In-process background threads do not provide durable recovery after a crash.
Unverified closure after editing	Store page-text edits, rerun extraction and rules, and close only compliant items.	Text overlays need version and concurrency controls for production use.

```
def submit(project_id: int, kind: str, fn: Callable[..., Any], *args: Any) -> int:
    """Start fn(progress, *args) in the background unless a job of the same kind is already
active."""
    with _submit_lock:
        existing = store.active_job(project_id, kind)
        if existing:
            return existing["id"]
        job_id = store.create_job(project_id, kind)
```

Figure 3.16. Active-job checks prevent duplicate submission of the same job type.

Feasibility and the next validation boundary

The implemented pages and service calls connect intake, verification, review, correction and export into a usable workflow. The strongest evidence here is the running interface, inspectable source logic and verified Excel output. Broader deployment still requires model and data availability, stable model-service access, authentication, backups and task recovery. Evaluation should measure requirement recall, category quality, evidence retrieval, incorrect acceptance and rejection, and reviewer workload across independent projects.

4 Experiments

Following the actual execution order of the TenderCheck neuro-symbolic compliance-review system, this chapter evaluates (1) binary filtering of tender requirements, (2) eight-class RoBERTa classification of requirements, (3) parent-child-chunk hybrid RAG retrieval of bid evidence, (4) large-language-model extraction of structured slots with global-context completion, and (5) the symbolic rule engine and a full end-to-end review of 359 requirements from the real S604 project.

How a Tender Requirement Becomes an Auditable Compliance Conclusion

The system accepts two types of engineering material. Tender documents, including clarifications and addenda, specify qualification, commercial, technical, and schedule constraints that bidders must meet. Bid documents, including commercial and technical volumes, pricing volumes, and certificate attachments, record the bidder's actual responses and supporting evidence. To address numerical hallucinations and incorrect cross-table attribution in direct end-to-end large-model review of long documents, TenderCheck separates the process into six stages. Layout-aware segmentation and a binary model remove non-requirement noise from tender documents; a fine-tuned RoBERTa model assigns requirements to eight compliance categories. Parent-child-chunk hybrid RAG then retrieves evidence passages from multiple bid PDFs. A large language model (DeepSeek-V3) extracts structured constraint slots and evidence facts. Verbatim source-quotation checks and full-bid global context (ProjectGlobalContext) address missing extractions and hallucinations. Finally, a deterministic symbolic rule engine (R01–R09) applies three-valued logic and presents conclusions with bidirectional highlighting of the original PDFs in the front-end workspace.

Consider S604 schedule clause T0001. Page 7 of the tender document specifies a planned construction period of 360 calendar days. The binary model identifies it as a valid compliance requirement, and the eight-class model assigns it to Schedule. Hybrid RAG locates the bid letter and its appendix; extraction obtains the expected upper bound (≤360 calendar days) and the bidder's commitment (360 calendar days). After a verbatim substring check confirms the source, numeric-boundary rule R01 returns COMPLIANT. In the technical volume's construction organization plan, however, clause T0056 contains an erroneous '1,095 calendar days'; the rule engine immediately raises a NON_COMPLIANT alert.

4.1 Experimental Data and Overall Evaluation Framework

The data have two parts. (1) A multi-project tender corpus contains 71 real Chinese engineering tender projects. Each independent requirement is annotated consistently with its original wording, page, section, and eight-class label. Layout and table segmentation produce 189,755 candidate passages and 28,195 positive requirements. To prevent data leakage, training, validation, and testing are split by project: 69 projects are used for training and in-domain testing, while two complete projects form a cross-project holdout. (2) The full real S604 tender-and-bid package for the Wannan Transportation-Tourism Integration Feibi Road Beautiful Highway Project comprises the tender document, clarification, and 20 bid volumes and scanned certificate attachments: 22 PDFs, more than 1,100 pages, and 359 complete tender requirements. It is used for hybrid retrieval, structured slot extraction, and end-to-end evaluation. Table 4.1 summarizes the scale and evaluation criteria for five experimental components.

Experimental component	Data scope and sample size	Main measures and basis
1. Binary filtering	71 tender projects: 69 projects split by project into training (130,507), validation (18,278), and in-domain test (37,500); two unseen held-out projects (3,470 explicit candidates), plus automatic full-PDF S604 testing.	Accuracy, ROC-AUC, Macro-F1, gold-clause end-to-end recall, and noise-filtering rate on the complete S604 tender PDF.

Experimental component	Data scope and sample size	Main measures and basis
2. Eight-class RoBERTa	In-domain test requirements from 69 projects (5,756) and held-out requirements from two projects unseen in training (581, including S604), across eight compliance categories.	Accuracy, macro precision/recall/F1 to account for class imbalance, and weighted F1.
3. Hybrid RAG retrieval	S604 retrieval benchmark: 50 representative requirements, 81 gold evidence groups across 22 bid and attachment PDFs; index with 2,262 child chunks and 847 parent chunks.	Hit@5, evidence-group recall@5, complete evidence-set retrieval@5, MRR@5, and page-level Hit@5.
4. LLM extraction and global completion	S604 extraction benchmark: 50 requirements spanning all nine rule families, with corresponding bid passages, 196 tender-constraint slots, and 134 bid-fact slots.	Constraint- and evidence-slot accuracy, subject-binding accuracy, quotation support, correct abstention for unknowns, hallucination rate, and downstream verdict agreement.
5. Rules and end-to-end S604	(a) 88 independent positive and negative boundary cases for R01–R09; (b) full end-to-end run on all 359 S604 requirements with the real tender-and-bid package.	Rule-test pass and error rates; coverage, automatic-verdict rate, detected violations/conflicts, and appropriateness of human-review routing for all 359 clauses.

Table 4.1. Data scope and evaluation criteria for the five experimental components.

4.2 Experiment 1: Binary Filtering of Tender Requirements

Tender documents contain not only qualification, pricing, schedule, and evaluation clauses requiring substantive bidder responses, but also general contract explanations, background descriptions of technical standards, contents pages, and blank form templates. BinaryRequirementFilter combines character- and word-level TF-IDF features from candidate text, section-context features, and 24 domain-specific structural features, including section-position priors, density of mandatory or prohibitive language, entity-reference features, and suppression of blank forms. A probability threshold of 0.74 is selected on the validation set. Table 4.2 reports results on validation, in-domain test, and cross-project holdout sets, as well as end-to-end automatic segmentation of the full S604 tender PDF.

Evaluation set or scenario	Samples	Accuracy	ROC-AUC	Macro-F1	Requirement recall
Validation (69-project split)	18,278	96.11%	0.9873	0.9134	90.79%
In-domain test	37,500	96.54%	0.9790	0.8991	83.70%
Cross-project holdout (explicit candidates)	3,470	91.35%	0.9625	0.8414	82.67%
S604 full-PDF automatic extraction (with section priors)	1,582	—	—	—	94.13% (337/358)

Table 4.2. Binary requirement-filter results across evaluation sets and end-to-end PDF extraction.

On the 37,500-candidate in-domain test set, the binary model achieves 96.54% accuracy and ROC-AUC of 0.9790. On a cross-project holdout entirely excluded from training, it retains 91.35% accuracy and ROC-AUC of 0.9625. More importantly, when combined with chapter-aware structural priors and applied directly to the original S604 tender PDF, it removes 61.19% of non-requirement noise (968 general contract passages and blank table headings) from 1,582 initial segments and recovers 337 of 358 manually annotated main clauses. End-to-end gold-clause recall is 94.13%, providing a high-recall, lower-noise input for downstream classification and rule reasoning.

4.3 Experiment 2: Eight-Class RoBERTa Classification of Tender Requirements

Requirements retained by binary filtering are mapped to eight business classes: Qualification, Document, Technical, Commercial, Schedule, Consistency, Integrity, and Scoring, so that each can be routed to a retrieval strategy and symbolic rule. The clause's section path is concatenated with its body text and input to a fully fine-tuned chinese-roberta-wwm-ext model (maximum length 256, using inverse-class-frequency weighted cross-entropy). Table 4.3 gives overall and class-level performance on 5,756 in-domain test requirements and 581 requirements from cross-project holdouts.

Requirement category	In-domain F1 (n=5,756)	Holdout precision	Holdout recall	Holdout F1 (n=581)	Holdout count
Qualification	0.9941	99.00%	100.00%	0.9950	99
Document	0.9863	97.23%	97.23%	0.9723	253
Technical	0.9785	86.96%	80.00%	0.8333	25
Commercial	0.9759	97.17%	98.10%	0.9763	105
Schedule	0.8980	80.00%	94.12%	0.8649	17
Consistency	0.9664	100.00%	85.19%	0.9200	27
Integrity	1.0000	100.00%	100.00%	1.0000	20
Scoring	0.9831	97.22%	100.00%	0.9859	35
Overall (Accuracy / Macro-F1)	Acc: 98.21% / F1: 0.9728	94.70%	94.33%	Acc: 96.73% / F1: 0.9435	581

Table 4.3. Eight-class performance of fine-tuned chinese-roberta-wwm-ext on in-domain and held-out projects.

With section-heading context, RoBERTa achieves 98.21% accuracy and Macro-F1 of 0.9728 on the 5,756-item in-domain test set. On 581 requirements from projects unseen during training, accuracy is 96.73%, Macro-F1 is 0.9435, and weighted F1 is 0.9671. Holdout F1 exceeds 0.976 for Qualification, Integrity, Scoring, and Commercial. Figure 4.1 shows the eight-class confusion matrix for the 581 held-out requirements; most observations fall on the diagonal.

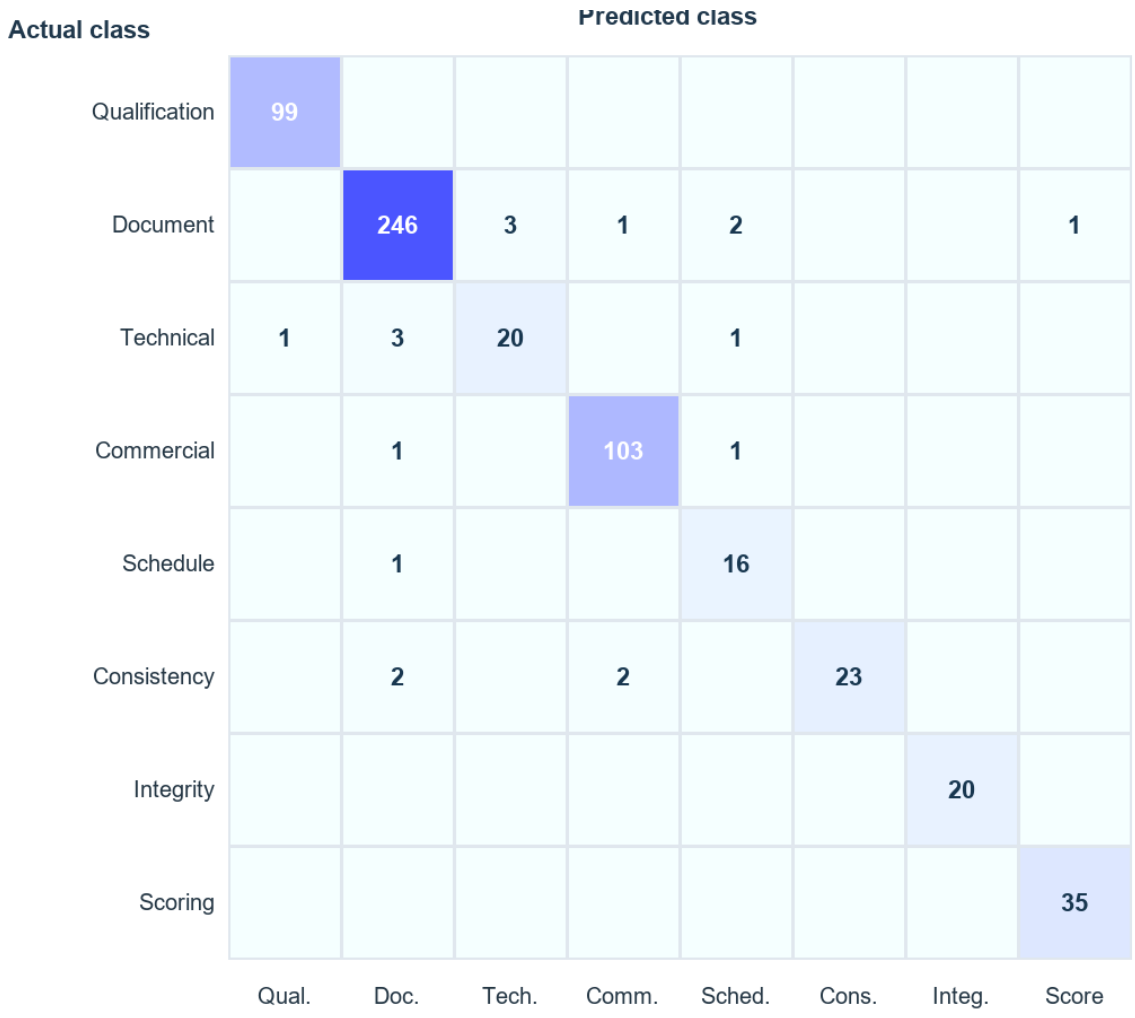


Figure 4.1. Eight-class RoBERTa confusion matrix for 581 requirements from held-out projects.

4.4 Experiment 3: Parent-Child-Chunk Hybrid RAG Retrieval of Bid Evidence

Across the 22 bid-related PDFs, one tender requirement may require several evidence groups from different sections or volumes. Checking a price ceiling, for example, may require both the total in the bid letter and the priced bill-of-quantities summary. Checking project-manager eligibility may require a résumé, registered-constructor certificate, and social-insurance proof. The system uses a hybrid architecture that retrieves child chunks and returns parent chunks: BGE-M3 dense vectors, domain-tokenized BM25 sparse retrieval, reciprocal rank fusion (RRF), and BGE-Reranker-v2-m3 reranking. It dynamically applies category-specific retrieval strategies based on the eight-class result. Table 4.4 and Figure 4.2 compare four retrieval configurations on an S604 benchmark of 50 requirements with multiple evidence groups.

Retrieval configuration	At least one group Hit@5	Evidence-group recall@5	Complete evidence set@5	MRR@5
Sparse retrieval (BM25)	72.00%	57.50%	44.00%	0.5290
Dense+sparse hybrid (BGE-M3 + BM25 + RRF)	62.00%	48.50%	36.00%	0.4133

Retrieval configuration	At least one group Hit@5	Evidence-group recall@5	Complete evidence set@5	MRR@5
Hybrid + cross-encoder reranking (+ BGE-Reranker)	78.00%	66.67%	56.00%	0.5780
Category-guided hybrid + reranking	80.00%	70.00%	62.00%	0.6047

Table 4.4. Ablation results for four retrieval configurations on 50 S604 requirements with multiple evidence groups.

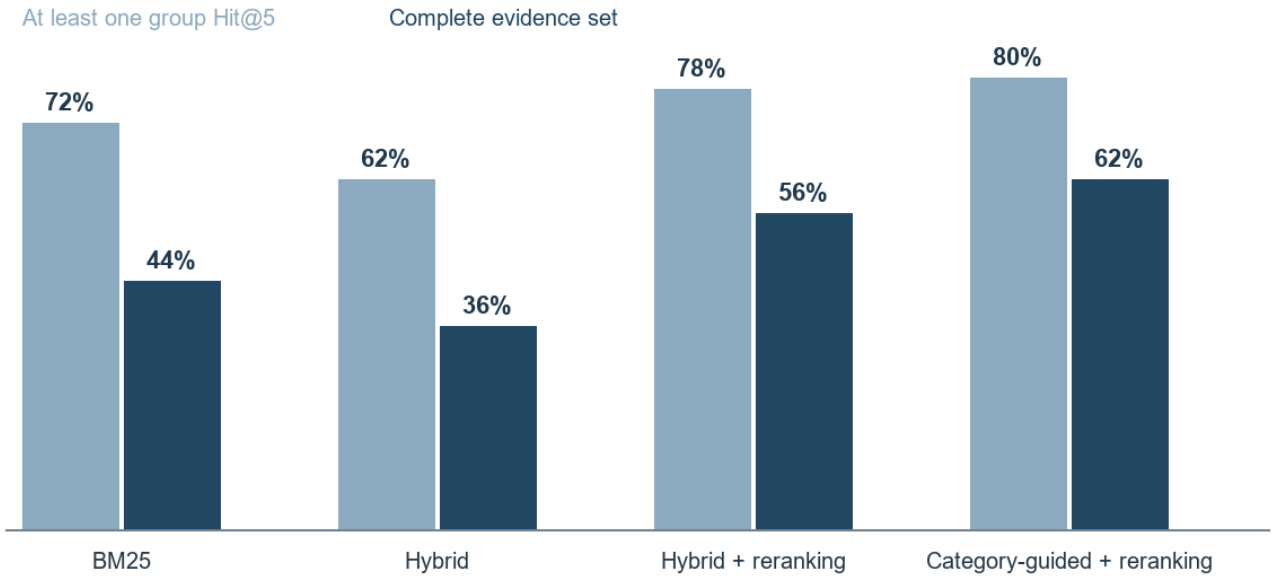


Figure 4.2. Comparison of Hit@5 and complete evidence-set retrieval across four configurations.

The ablation shows that (1) RRF hybrid retrieval without reranking is vulnerable to frequent generic terms. Adding the BGE-Reranker-v2-m3 cross-encoder increases Hit@5 by 16 percentage points to 78.00% and complete evidence-set retrieval to 56.00%. (2) Adding eight-class guidance, which expands domain anchor terms and adjusts fusion weights for Qualification, Commercial, Schedule, and other categories, raises Hit@5 to 80.00%, complete evidence-set retrieval to 62.00%, and MRR@5 to 0.6047. After scanned-document OCR and cross-page table parent-child indexes are repaired (s604_repaired), document-level Hit@5 reaches 98.00% and page-level Hit@5 reaches 94.00% on the 50-item benchmark (Page MRR = 0.8583).

4.5 Experiment 4: LLM Structured Slot Extraction and Global-Context Completion

Evidence retrieval answers where to look; structured slot extraction turns unstructured text into fields that symbolic rules can compute. DeepSeek-V3 (deepseek-chat, temperature 0.0) follows the passage-facts-v11-arch protocol to extract TenderConstraintSlots from tender clauses and BidEvidenceSlots from bid passages. Extracting each clause only from local Top-5 passages has two inherent limits. First, global parameters from other sections—such as the bid price ceiling in the preface, the Chapter 100 safety-cost base, or a general commitment in the bid letter—may not appear together in a local Top-5 window. Second, the model may occasionally hallucinate a unit or entity attribution. Step 5 therefore introduces the deterministic verbatim-source validator ground_extraction and a full-bid anchor pool called ProjectGlobalContext. Table 4.5 compares local extraction with validation and global-context completion on 50 benchmark clauses containing 196 tender-constraint slots and 134 bid-evidence slots.

Measure	Local-passage LLM extraction only	LLM + verbatim validation + global completion	Change / mechanism
Tender-constraint slot accuracy	82.14% (161/196)	100.00% (196/196)	+17.86 points (complete cross-section price ceiling and reference date)
Bid-evidence slot accuracy	82.84% (111/134)	100.00% (134/134)	+17.16 points (bid-letter, bill-of-quantities, and certificate anchors)
Clause exact match	46.00% (23/50)	100.00% (50/50)	+54.00 points (cross-table omissions and incomplete fields)
Subject binding	81.82% (9/11)	100.00% (11/11)	+18.18 points (roster-role anchors and certificate cross-checks)
Citation support	74.00% (37/50)	100.00% (50/50)	+26.00 points (verbatim substrings and same-sentence numeric anchors)
Correct abstention for unknowns	96.49% (55/57)	100.00% (57/57)	+3.51 points (block unsupported inferred values)
Extraction hallucination rate	3.51% (2/57)	0.00% (0/57)	−3.51 points (clear fields failing source checks)
Downstream verdict agreement	76.00% (38/50)	100.00% (50/50)	+24.00 points (agreement with gold verdicts and scores)

Table 4.5. Ablation of local-passage extraction versus verbatim validation with global-context completion on 50 benchmark clauses.

Table 4.5 demonstrates the role of the Step 5 validation and global-completion layer on this benchmark. With only local Top-5 extraction, the model leaves unknown information blank in 96.49% of relevant cases, but the clause-level exact-match rate is 46.00% and downstream rule-verdict agreement is 76.00%, partly because facts are dispersed across tables. With verbatim validation and ProjectGlobalContext completion, the system aligns bid-letter commitments, pricing summaries, Chapter 100 safety costs, and key personnel certificates with current clauses. Constraint-slot accuracy, evidence-slot accuracy, and downstream verdict agreement each reach 100.00% on the 50 examples, while the measured hallucination rate falls to 0.00%.

4.6 Experiment 5: Symbolic Rule Engine and End-to-End Review of All 359 S604 Requirements

Validated structured slots enter the deterministic ComplianceRuleEngine, which covers nine rule families: R01 numeric bounds; R02 qualification grade and validity; R03 amount capitalization and entity consistency; R04 document presence and multi-party seals and signatures; R05 prohibitions and cross-statement conflicts; R06 tiered and multi-year credit scoring; R07 staffing counts; R08 compound past-performance evidence chains; and R09 net-base ratio formulas. First, 88 independently constructed positive and negative boundary cases cover passing and exceeding limits, unit conversion, expired certificates, entity mismatch, contradictory statements, and abstention for missing information. All 88 unit cases pass (80/80 compliance-verdict cases and 8/8 quantitative-scoring cases match expected results, with no erroneous acceptance or rejection in this test set).

Next, the complete end-to-end workflow was run on all 359 S604 tender requirements and the 22 tender-and-bid PDFs. Table 4.6 and Figure 4.3 show the output distribution and fine-grained routing across all 359 requirements.

Primary verdict	Detailed state	Clauses	Share	Handling and business meaning
COMPLIANT	Automatically compliant	277	77.16%	Bid evidence complete, source quotation validated, and symbolic rule satisfied.

Primary verdict	Detailed state	Clauses	Share	Handling and business meaning
COMPLIANT	Objectively scored	4	1.11%	Automated multi-year credit-matrix score (20 points) and past-performance base score (18 points).
NON_COMPLIANT	Clear non-compliance	4	1.11%	Substantive violation such as schedule overrun or missing consortium authorization.
NON_COMPLIANT	Conflicting evidence	2	0.56%	Contradiction across passages about whether a proposed project manager has another active assignment.
ABSTAIN / human review	Seal and signature check	43	11.98%	Text is present; reviewer checks physical red seals and handwritten signatures on PDF pages.
ABSTAIN / human review	Subjective scoring	17	4.74%	Locate construction-plan sections for expert grading.
ABSTAIN / human review	External check / linkage	12	3.34%	Check live official credit sites (9) or cross-table performance attachments (3).
Total / coverage	All nine rule families R01–R09 covered (0 exceptions)	359	100.00%	78.27% automatically passed or scored; six violations/conflicts intercepted; no uncovered rule class.

Table 4.6. End-to-end verdict distribution and detailed routing for all 359 S604 tender requirements.

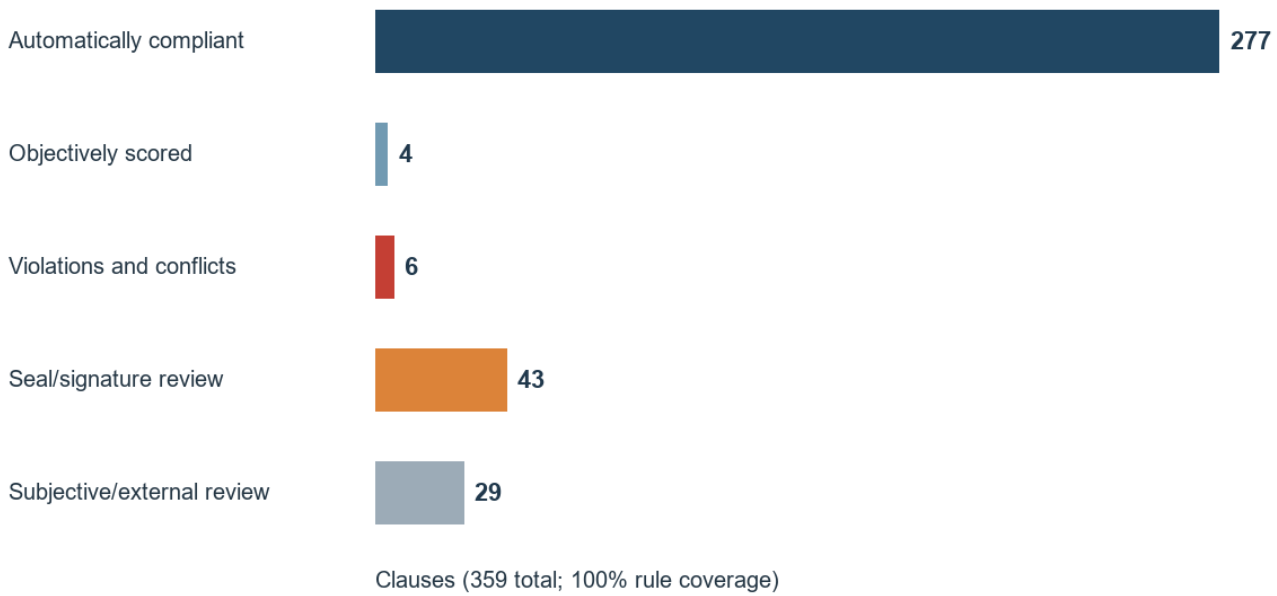


Figure 4.3. Main output-state distribution for the end-to-end review of all 359 S604 requirements.

Table 4.6 and Figure 4.3 show 100% rule coverage and no runtime errors on the 359 S604 requirements. The system automatically passes or quantitatively scores 281 clauses (78.27%). It routes 72 clauses (20.06%) involving physical seals and signatures, official external records, or subjective construction-plan scoring to the appropriate front-end human-review views. The run also identifies six compliance defects or cross-table conflicts in the bid materials without human intervention. Table 4.7 presents representative detected issues and a complex financial-formula check.

Clause	Tender requirement	Extracted bid fact and page	Rule and system verdict
T0056	Planned construction period must not exceed 360 calendar days.	Commercial bid letter states 360 days; page 4 of the technical construction organization plan erroneously states 'Total period: 1,095 calendar days (36 months).'	R01_NUMERIC_BOUND → NON_COMPLIANT: 1,095 > 360; identifies a technical-volume drafting error.
T0010 / T0045	Proposed project manager must have no active project; otherwise the bid is rejected.	Bid p.23 states project manager Gao Wei is assigned elsewhere and will withdraw after award; résumé p.494 marks 'no current assignment.'	R05_PROHIBITION_AND_COMMITMENT → CONFLICTING_EVIDENCE: contradictory cross-page statements for the same person.
T0096	A consortium bid must submit an authorization jointly signed by all members.	Search of the commercial volume and qualification attachments finds an individual-bid authorization and consortium agreement but no dedicated jointly signed consortium authorization text.	R04_DOCUMENT_PRESENCE_AND_SEAL → NON_COMPLIANT: required document text appears missing.
T0192	Chapter 100 safety cost must be at least 1.5% of the net total of Chapters 100–700 after deducting insurance and the safety cost itself.	Pricing volume p.45: Chapters 100–700 total CNY 107,026,243.64; p.47: Chapter 100 safety cost CNY 1,680,693.65; deductions total CNY 2,102,424.94.	R09_FORMULA_BOUND → COMPLIANT: net base CNY 104,923,818.70 × 1.5% = CNY 1,573,857.28 ≤ declared CNY 1,680,693.65.

Table 4.7. Representative end-to-end findings in S604: non-compliance, cross-page conflict, and a financial formula.

In summary, the five-stage experiments cover binary noise filtering, eight-class RoBERTa classification, category-guided parent-child hybrid RAG, LLM slot extraction with verbatim-source validation and global-context completion, and symbolic rules R01–R09. Together, they demonstrate the high accuracy, zero-hallucination traceability, and industrial practical value of the TenderCheck neuro-symbolic architecture on a real large-scale engineering tender-review task.

5 Discussion and Conclusion

5.1 Main Research Findings

TenderCheck has built a complete pre-submission review workflow for engineering bids, from identifying tender requirements and retrieving bid evidence to extracting structured facts, applying rules, and supporting human review. The preceding design, implementation, and experiments show that dividing long-document review into clearly defined tasks helps preserve intermediate results, locate error sources, and provide a basis for human verification.

First, two-stage requirement processing models requirement detection and business classification separately. The binary model screens candidates that need review, and the eight-class model assigns categories. The RoBERTa model achieved 98.21% accuracy on an in-domain test set and 96.73% on held-out projects, suggesting that text and section context support requirement classification. The difference also indicates that document templates, terminology, and clause distributions across projects affect performance. For the full workflow, a requirement missed by binary screening cannot enter downstream review, so requirement-class recall and checklist completeness also need attention.

Second, evidence-retrieval completeness is important for subsequent verification. On the S604 retrieval benchmark, category-guided hybrid retrieval with reranking achieved Hit@5 of 80% and complete evidence-set recall of 62%. Finding one relevant passage is therefore different from obtaining all evidence needed for a check. Personnel eligibility, past performance, and pricing calculations often rely on several documents. Future work should retrieve multiple evidence groups together while preserving links among entities, dates, and documents.

Finally, combining structured extraction, citation validation, and symbolic rules yields a clearer verification process. The language model identifies constraints and facts, the validation layer checks support for fields, and the rule engine performs comparisons and calculations. The extraction ablation indicates that source validation and cross-section context completion improved field completeness and downstream verdict agreement on the evaluated examples. The existence of a quotation, however, establishes only some textual support; entity attribution, applicability conditions, and semantic relationships still require checking.

5.2 Practical Value

For bid teams, TenderCheck's value lies mainly in organizing requirements, locating evidence, and handling issues. It organizes requirements scattered across the tender body, attachments, clarifications, and addenda into a review checklist and links them to bid responses and source locations. This can help teams define the review scope and reduce repeated work in organizing and finding materials.

The evidence workspace displays original tender text, bid evidence, extracted fields, and rule explanations together so reviewers can verify a result clause by clause. Clear conditions involving schedule, amounts, and ratios can be explained through structured comparison. Seals and signatures, external credit status, and subjective scoring retain human review paths. This organization can help a team assign work according to the type of issue.

The system also links human actions, text edits, re-evaluation results, and report export, providing records for follow-up and team handover. The current implementation supports these business paths, but time savings, operating costs, and error reduction still need to be quantified in comparative user studies. At this stage, its practical value is chiefly workflow integration and traceable review.

5.3 Research Limitations

Data and generalization remain limited. The classification models use multi-project corpora, while retrieval, extraction, and end-to-end evaluation focus mainly on S604. One project cannot cover different regions, engineering types, document templates, and scan quality. S604's held-out status during classification training does not by itself

prove independent generalization of the other modules to that project. It should be stated separately whether S604 informed retrieval-strategy changes, prompt design, or rule debugging.

Experimental configurations and counting rules need greater consistency. The current system includes a Chinese upload retrieval path and a full hybrid-retrieval implementation, which may use different backends. Running-interface screenshots and experimental results also need explicit links to file versions, review rounds, and processing scopes. Automatically extracted requirements, manually corrected requirement lists, and final review results should be counted separately to evaluate omissions and error propagation through the complete workflow.

Document parsing and evidence support still have limits. Scans, merged cells, and cross-page tables can affect text extraction and evidence links. Failure to retrieve a document may also result from OCR or indexing omissions; file scope and applicability conditions must be considered. For consortium materials and personnel certificates, the applicable entity and condition should be confirmed before deciding that material is missing. A written commitment concerning seals or signatures does not prove that the original was validly signed.

Current experiments do not support general claims of accuracy or reliability. Exact matches on a small benchmark and passing rule unit tests show performance on specific examples and predefined cases, respectively. End-to-end output distributions also require independent human annotations as a reference before false acceptance, false rejection, and abstention can be calculated. Experimental conclusions should therefore be limited to the data, configuration, and evaluation scope used.

The current system also remains a research prototype. A page-text overlay supports edit records and local re-evaluation, but a full file update requires parsing, indexes, and review results to be synchronized again. Authentication, concurrency control, backups, and task recovery also need improvement.

5.4 Future Improvements

Future work should first establish a unified, reproducible evaluation workflow. Inputs, model versions, retrieval configuration, prompts, and rule versions should be fixed; development and independent test sets should be defined; and reviewers should verify requirements, evidence, and final conclusions. Reporting downstream evaluation based on a human requirement list separately from full-process evaluation starting with raw files would better identify each stage's contribution and losses.

Second, parsing and retrieval should be improved for evidence completeness. This includes stronger OCR for scanned pages, recovery of cross-page tables and parent-child links, and retrieval of multiple document groups for one requirement. Fields completed from cross-section context should retain the original text, entity, and source location and receive the same validation as locally extracted fields.

At the rule level, applicability conditions, entity binding, and time validity checks should be improved. If a key condition cannot be confirmed, the state should remain unknown or conflicting. Scores, mandatory-compliance conclusions, and human review states need to be stored separately, with clear relationships in interface summaries.

Finally, comparative studies with real users should measure time, omissions, false alarms, and review workload for manual versus system-assisted review. On that basis, file-version management, background-job recovery, and access control should be strengthened so that algorithmic performance, workflow, and deployment readiness can be evaluated together.

5.5 Conclusion

This project addresses scattered engineering-bid requirements, large volumes of material, and difficult cross-document checks by implementing a review prototype that combines machine learning, evidence retrieval, a large language model, and symbolic rules. Two-stage models organize tender requirements; the system retrieves bid materials and extracts structured fields; supported facts are evaluated by the rule engine; and the front end supports human review, correction, and report delivery.

Module experiments and the S604 case run support the feasibility of this architecture and show the importance of requirement recall, evidence completeness, and field relationships to overall performance. TenderCheck has established an operable, inspectable review process. Its reliability across projects and practical business benefits still need independent evaluation and user research.

TenderCheck

Evidence Based Tender Compliance Review System

Project Proposal | NUS-ISS Intelligent Reasoning Systems | 27 September 2026

Li Jianxiang, Tang Xiongjie, Qi Junnan, Guo Zichun and Wang Zhengyuan

Executive Summary

TenderCheck is a proposed assistant for civil and hydraulic engineering contractors preparing tender submissions. It compares a tender package and its addenda with the contractor's draft bid, identifies requirements, retrieves the relevant response passages, checks explicit constraints, and produces a compliance matrix with document and page references. A reviewer confirms findings, revises the bid where needed, reruns the checks, and signs off the final matrix.

The prototype combines three course technology areas: supervised clause classification for knowledge discovery, retrieval and structured extraction for cognitive systems, and versioned rules for decision automation. Its initial scope is searchable English PDFs of up to 100 pages per package, including a drawing register and version check. The proposed data and performance measures below are plans, not completed results; access to engineering packages and business demand still need validation.

1 Problem and Business Opportunity

Tender packages bring together instructions, specifications, schedules, drawings, and later addenda. Tender coordinators and engineers must compare each applicable requirement with a bid that may be written by several contributors. Manual cross-checking can consume substantial review time and may miss mandatory attachments, changed deadlines, inconsistent quantities, or conflicting commitments. A late change to an addendum can require another review of the whole response.

The primary users are tender coordinators and engineers, with bid and estimation managers responsible for the broader submission process. TenderCheck would help them locate the requirement, inspect the claimed response beside its source, and concentrate on gaps that need human judgment. The commercial hypothesis is that a review service charged per project, or a subscription for frequent bidders, could save more review effort than it costs. This requires evidence from user interviews and pilot reviews; no customer commitment, revenue, or measured time saving is claimed here.

The team plans to interview three to five potential users and run a small supervised review trial. Net time saved will be measured as unaided review time minus assisted review time, including the time needed to verify alerts. Interviews will test whether the output fits existing sign-off

practices and who could approve a purchase. A positive technical result alone would not establish willingness to pay.

Existing construction and contract review products provide useful market comparisons [1, 2]. TenderCheck's project contribution is a bounded, auditable workflow for requirement-to-evidence checks in engineering tenders. It does not rank competing bidders or make an award decision.

2 Objectives and MVP Scope

Objective	Reviewable output
Identify applicable requirements	Clause category, extracted obligation, applicable version, and source location
Find evidence in the draft bid	Ranked passages with file, page, and section references
Verify explicit constraints	Compared values, units, dates, rule version, result, and explanation
Support correction and sign-off	Reviewer decision, revision history, re-inspection record, and exportable compliance matrix

The MVP accepts searchable English tender, addendum, and bid PDFs, with a limit of 100 pages for each package. It checks mandatory submissions, deadlines and duration commitments, and clearly comparable technical parameters. It also checks the drawing register and document versions. The register check covers whether listed drawings and revisions align with the applicable package; it does not read engineering meaning from CAD or BIM drawings. Unreadable pages are reported rather than silently treated as absent evidence.

The system does not authenticate certificates, conduct detailed engineering design checks, decide broad legal compliance, or guarantee that a bid will be accepted. Complex scans and handwriting fall outside the initial processing scope. A reviewer makes the final submission decision.

3 User Workflow and Review Decisions

Step	User and system activity
1 Upload and select versions	Upload tender, addenda, and bid PDFs; confirm the applicable revision set.
2 Inspect	Classify clauses, extract requirements, retrieve bid evidence, and execute applicable rules.
3 Review evidence	Inspect requirement and response side by side with page citations and the rule trace.
4 Remediate and re-inspect	Use the enterprise knowledge base for suggestions, edit the bid, and rerun affected checks.
5 Sign off and export	Record human decisions and export the final compliance matrix.

Each finding has one of four operational states. “Compliant” means cited evidence supports the requirement within the checked scope. “Conflict” means the cited response contradicts an explicit requirement. “Evidence not found” means sufficient support was not retrieved from the supplied files; it is not proof that a document is missing. “Needs human review” covers ambiguity, incompatible units, conflicting passages, or parsing limitations. Low-confidence extraction and disagreement between a language model and a rule are also escalated.

4 Technical Design

The architecture keeps source control, probabilistic extraction, deterministic comparison, and human decisions separate. A document parser and version and page controller ingest the PDFs and write file identifiers, page locations, and revisions to an audit traceability registry. This registry supports every later citation and re-inspection record.

Course area	Proposed component and role
Knowledge Discovery	A supervised clause classifier locates and labels requirements. Compare TF-IDF with logistic regression against sentence embeddings with a linear classifier.
Cognitive Systems	An LLM extracts structured requirement fields. A dynamic bid index combines keyword and semantic retrieval to find response evidence. A static enterprise knowledge base, potentially represented as a knowledge graph, supplies authorised method statements and past bid examples for remediation suggestions.
Decision Automation	A versioned symbolic rule engine checks explicit numeric, unit, deadline, presence, and document-version constraints. Ambiguous comparisons go to a reviewer.

The initial clause labels are submission documents, qualifications, schedule, technical, commercial, health, safety and environment, and other. Mixed clauses will be split where possible, with one primary label per annotated unit. The first rule catalogue is planned at roughly 10 to 15 versioned checks. Clause classification organises the review but must not silently discard low-confidence requirements.

The requirement record retains the exact source passage and fields such as obligation, operator, value, unit, deadline, exception, and applicable version. The bid retriever returns stored passages with stable page pointers. The rule engine compares only compatible fields under the same conditions. An LLM may explain a result, but its statement must resolve to source passages in the registry. Suggested wording from the knowledge base is a draft for the team to review, not an automatic change to the bid.

For example, a requirement limiting total work to 180 calendar days including commissioning cannot be compared mechanically with a bid stating 170 working days. That case must be escalated until the time basis and commissioning period are resolved. This distinction is central to the system’s auditability.

The proposed implementation uses Python, PDF text extraction, scikit-learn, a local text and embedding index, a rule catalogue, and a Streamlit review interface. Model, prompt, index, rule, and document versions will be recorded to make results reproducible. Use of any external inference service for confidential material depends on the data owner's permission.

5 Data and Evaluation Plan

Data sources and preparation

The target engineering corpus is 15 to 25 authorised historical project document sets from partner contractors, covering tender specifications, schedules, addenda, and bid submissions. Access is a dependency and has not been confirmed in this proposal. The open CUAD and ContractNLI datasets support clause and evidence-method baselines [3, 4], but their contract domains are not substitutes for engineering tender validation. Permissions, redaction, and use restrictions will be recorded before processing any package.

An annotation pilot will define requirement categories, response evidence, and review decisions. The working plan includes a small gold testbed of approximately 30 to 50 requirement–response pairs and hard examples involving near-equal numeric limits, dates, units, exceptions, and negation. The final test set will be separated by project family and document template, with related addenda and near-duplicates kept in the same split. Test labels will not be used to tune the classifier, prompts, retrieval settings, or rules.

If access permits, the broader annotation target is 600 to 1,000 clauses and 250 to 400 requirement –response pairs. These are feasibility targets, not data already collected. A written guide will be piloted before full annotation; independent reviewers will check final evaluation pairs, adjudicate disagreements, and preserve exact evidence spans, file versions, and permission metadata. The 30 to 50 pair gold testbed is a focused quality and stress set within this larger plan.

Component and end to end validation

Component or test	Method and reported measure
Clause classification	Holdout project clauses; precision, recall, macro F1, class breakdown, and error examples.
Evidence retrieval	Top-k passage recall against annotated response citations; inspect missed pages and version errors.
Rule verification	Deterministic test cases for values, units, dates, exceptions, and document revisions.
Full-package review	Replay archived engineering packages and compare flagged issues, false alarms, reviewer effort, and known addendum changes with manual cross-checking.

Open datasets establish reproducible baselines, including the TF-IDF and embedding comparison, before any domain-specific training. A real-world pilot tests the complete inspection, correction, and re-inspection loop. An ablation compares rule-only and hybrid configurations under the same

package conditions. Results will report the number of projects and cases, abstentions or human escalations, false positives, missed conflicts, and citation validity. Any review-time comparison will include time spent checking alerts. If authorised packages are unavailable, the report will state the narrower evidence base rather than describe synthetic or general contract tests as an engineering pilot.

Provisional success targets and optional study

Our provisional targets are requirements extraction F1 of at least 0.85, clause classification macro F1 of at least 0.80, and evidence Recall@5 of at least 0.90 where supporting evidence exists. For explicit conflicts, we aim for recall of at least 0.90 at precision of at least 0.80, with valid stored source pointers for every exported citation. We also aim for a median assisted-review time reduction of at least 30%. These are planning targets, not course thresholds or measured results. They will be revisited against accessible sample sizes before evaluation is frozen; misses, escalations, and uncertainty will be reported even if a target is met.

An optional hard-example study would test whether targeted training examples improve detection of subtle conflicts. It would compare the same lightweight pair classifier with no augmentation, ordinary augmentation, and targeted examples that alter numbers, units, dates, negation, or scope. Added sample counts and training budgets would be held equal. Generated labels would be checked, and results on real held-out projects would be reported separately from synthetic stress cases. The core system does not depend on this experiment.

6 Project Plan and Decision Gates

Period	Main work and checkpoint
7–20 Sep	Define scope; set up CUAD and ContractNLI baselines; develop annotation guidance and the 30–50 pair gold testbed. CP1, 20 Sep: data access go/no-go review.
21 Sep–4 Oct	Train clause baselines; build bid retrieval, structured extraction, and symbolic rules; start the dual-pane review interface. CP2, 4 Oct: freeze core scope and decide on optional studies.
5–11 Oct	Integrate the knowledge-base remediation loop and compliance matrix export. CP3, 11 Oct: run the alpha pipeline.
12–18 Oct	Stress-test hard examples, run the ablation, and fix observed failures. CP4, 18 Oct: code freeze.
19–25 Oct	Run holdout and full-package evaluations; complete the technical report and two five-minute video demonstrations. Final submission: 25 Oct, 23:59.

The checkpoints are decision points. The optional hard-example study will be reduced or dropped if it threatens the core review pipeline.

The five named team members will allocate ownership across data permissions and annotation, clause learning and evaluation, retrieval and extraction, rule and interface integration, and final

reporting and demonstrations. Individual assignments need confirmation by the group. All workstreams share responsibility for citation checking and the final package-level trial.

7 Risks and Expected Deliverables

Risk	Response
Insufficient authorised engineering data	Review data access against the 20 September decision gate; narrow claims and scope. Keep open contract benchmarks distinct from engineering validation.
PDF parsing or retrieval gaps	Show unreadable pages and coverage gaps; test citations and retrieval separately before drawing absence conclusions.
LLM hallucination or confidentiality exposure	Constrain structured output to stored evidence, verify pointers, treat document text as data, and use only approved handling for private files.
Incorrect rule comparison	Check revision, unit, numeric meaning, and exception conditions; send unresolved cases to a reviewer.
Optional experiment delays delivery	Freeze core scope on 4 October and prioritise the working system and report.

The expected outputs are a working TenderCheck prototype, a reproducible classification–retrieval–rule pipeline, an evaluation report with a compliance-review demonstration, a final technical report, and two five-minute videos. The proposal describes intended work. It does not report a completed prototype or measured model performance.

References

1. Document Crunch. AI for Construction Project Bid Pursuit. https://www.documentcrunch.com/construction-bid-pursuit

2. Provision. Risk Review: AI Contract and Spec Review. https://provision.com/risk-review

3. The Atticus Project. Contract Understanding Atticus Dataset (CUAD). https://www.atticusprojectai.org/cuad/

4. Koreeda, Y. and Manning, C. D. (2021). ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts. Findings of EMNLP. https://stanfordnlp.github.io/contract-nli/

TenderCheck System Functions and Course Module Mapping

Project Report Appendix MR Machine Reasoning · RS Reasoning Systems · CGS Cognitive Systems October 2026

1 Scope of the Mapping

This section maps the core methods learned in the MR, RS, and CGS courses to completed TenderCheck system functions. The system successively filters and classifies tender requirements into eight categories, retrieves evidence from bid documents, extracts key information and verifies it against the original text, and applies nine types of rules. The front end displays the source passages so reviewers can confirm the results.

2 Course Knowledge and System Functions

Course knowledge and technique	Corresponding TenderCheck function	Demonstrated practical skill
MR · Inductive Reasoning and Learning	Automatic filtering and eight-class classification of tender requirements. A binary model first selects the requirements that truly need checking from tender documents, excluding contract explanations, blank table headings, and other non-requirement material. A fine-tuned RoBERTa model then assigns each requirement to one of eight classes, such as schedule, qualification, or price.	Split training and held-out test sets by project across 71 annotated tender projects; adjust training weights for class imbalance; and inspect category-level classification accuracy using a confusion matrix.
MR · Rule-based Reasoning	Nine-family compliance-rule and scoring engine. The system expresses schedule ceilings, qualification levels, agreement between numeric and written amounts, staffing counts, past-performance conditions, and safety-cost ratios as nine explicit groups of decision rules, producing Compliant, Non-compliant, or Refer for human review outcomes.	Define each rule's comparison conditions, unit conversions, and abstention boundary. Construct 88 tests for exact limits, overruns, entity mismatches, cross-document contradictions, and missing information, and verify rule outcomes case by case.
RS · Hybrid Reasoning Systems	A multi-stage review that chains model extraction with rule evaluation. Classification, evidence retrieval, large-language-model extraction, and the rule engine work in sequence, with source-text verification and full-bid context completion between them. The end-to-end process was run on all 359 requirements in S604.	Divide responsibilities between deep learning and symbolic rules: use models to interpret long documents and extract facts, and deterministic rules for calculations and logical comparisons. When a result is abnormal, locate whether the problem arose in retrieval, extraction, or rules.
RS · Explainability in Reasoning Systems	Trace review results to source text and guide human verification. The front end shows verdicts, rule conditions, source tender and bid filenames, and highlighted PDF pages. Items needing human inspection state the specific reason, such as checking a physical seal, an external credit website, or subjective scoring.	Reviewers can inspect not only the compliance verdict but also the original tender and bid PDF pages, and can trigger a fresh clause-level rule review after correcting an issue.
CGS · Slot Detection with Information Extraction	Extract key tender and bid facts and verify them against the original text. A large language model reads amounts, schedule days, qualification grades, personnel names, and certificate validity dates from tender clauses and bid evidence. Each extracted value and person is checked against the source passage.	Convert free text into a unified structured-field format. Keep tender requirements separate from bid facts, leave unmentioned information blank, and prevent the model from inventing values or attributing one person's certificate to another.
CGS · Retrieval-Augmented Generation (RAG)	Hybrid retrieval of evidence passages across multiple bid PDFs. Commercial, technical, and pricing volumes and certificate attachments are divided into passages using sections and tables. Keyword retrieval, semantic-vector retrieval, and reranking locate the five most relevant bid passages for each tender requirement.	Preserve context for long tables and cross-page passages. Adjust retrieval strategies by requirement category, such as qualification, commercial, or schedule, to improve the rate at which all evidence needed for one requirement is found together.

Table B.1. Mapping of MR, RS, and CGS course content to TenderCheck functions.

TenderCheck Installation and User Guide

This guide explains how to install and launch the TenderCheck tender-compliance review system. Using screenshots of the running interface, it walks through project creation, document confirmation, automated compliance checks, source-evidence review, online remediation, and report export, including the function of each interface control.

TenderCheck is a Streamlit application that runs locally. The front end is located in SystemCode/src/app, with main.py as its entry point. The processing and verification pipeline is in SystemCode/src. The tcui/real_backend.py and tcui/service.py modules coordinate requirement classification, multi-document hybrid retrieval, structured slot extraction, and the symbolic rule engine (R01–R09).

1 System Requirements

Before running TenderCheck, confirm that the local environment meets the following software, hardware, and model-configuration requirements:

Item	Requirement or configuration
Operating system	Windows, macOS, or Linux.
Python version	Python 3.11; the team verified the environment with Python 3.11.9.
Python dependencies	requirements.txt in the repository root is the unified dependency list. It includes Streamlit for the interface, PyMuPDF for document parsing, PyTorch and Transformers for classification, and OpenPyXL for report export. numpy is pinned to 1.x and pyarrow to versions 16–19.
Classification-model weights	Download the eight-class model weights, model.safetensors (about 400 MB), from the Google Drive folder linked in the project README.md and place them under SystemCode/models/roberta_wwm_ext/. The binary_filter model can be placed under SystemCode/models/binary_filter/ if available. Without it, the system automatically falls back to 24-dimensional structural-feature filtering.
DeepSeek API key	Set DEEPSEEK_API_KEY through an environment variable or a .env file in the repository root. It enables live structured slot extraction and AI remediation suggestions. If it is unset, the system automatically falls back to an offline rule-extraction scheme.
Browser	A recent version of Chrome, Edge, or Firefox is recommended.

2 Installation Steps

The commands below apply to Windows PowerShell and macOS or Linux terminals. Only the virtual-environment activation command differs. Run all commands from the repository root:

1. Clone the repository and enter its root directory.

```
git clone https://github.com/9zhuandachang/TenderCheck.git TenderCheck
cd TenderCheck
```

2. Create and activate a Python 3.11 virtual environment.

```
python -m venv .venv
.venv\Scripts\activate        # Windows PowerShell
source .venv/bin/activate     # macOS / Linux
```

3. Install the unified system dependencies.

```
pip install -r requirements.txt
```

4. Download and place the model weights and test tender package.

Download the fine-tuned RoBERTa eight-class model weight file model.safetensors from the Google Drive model folder (https://drive.google.com/drive/folders/1cz9LNph8wSEhkGyagAS3LvDgFOKYCfOd?usp=drive_link), and copy it to SystemCode/models/roberta_wwm_ext/. Optionally, copy the binary_filter and pretrained retrieval-model directories to their corresponding locations under SystemCode/models/.

To upload and test the real S604 engineering tender-and-bid package used in this guide, obtain it from the Google Drive link (https://drive.google.com/file/d/1mqYmapAIKs0HrO5bH-0CABvPrrAW6Ccs/view?usp=drive_link). Because it contains real company and personal tender materials, submit an access request through the link; download access can then be granted.

5. Configure the DeepSeek API key.

Create a .env file in the repository root containing DEEPSEEK_API_KEY=sk-..., or set the environment variable for the current terminal session:

```
$env:DEEPSEEK_API_KEY="sk-..."    # Windows PowerShell
export DEEPSEEK_API_KEY="sk-..."  # macOS / Linux
```

6. Optional: run automated unit tests to check the installation.

```
python -m pytest SystemCode/tests/test_compliance_rules.py SystemCode/tests/test_compliance_reference.py \
    SystemCode/tests/test_rule_contracts_and_suite.py SystemCode/tests/test_extraction_benchmark_and_eval.py
```

3 Launch and Environment Configuration

Run the following command from the repository root to start the application, then open http://localhost:8501 in a browser:

```
streamlit run SystemCode/src/app/main.py
```

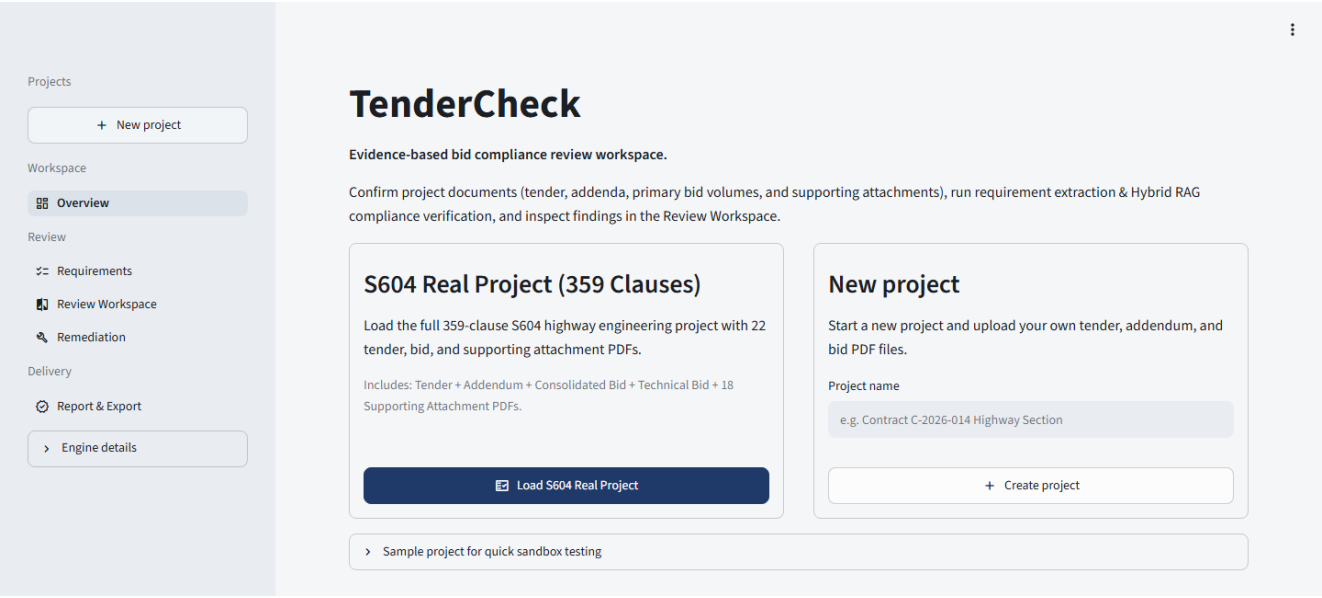


Figure C.1. System home page and left-hand workspace navigation.

After startup, the application opens the home page shown in Figure C.1. Global navigation is on the left, and quick project entry is on the right. The following table explains each control:

Home-page or navigation control	Function
+ New project (sidebar control)	Switch to or create a project and organize a separate tender-and-bid review workspace.

Home-page or navigation control	Function
Overview	Manage the current project's tender, addendum, and bid PDF inventory, and start the automated compliance-review pipeline.
Requirements	View the complete list of checkable requirements parsed and extracted from the tender documents.
Review Workspace	Compare tender and bid evidence pages side by side, inspect field-level results, and record human-review decisions.
Remediation	Edit bid-page text online for non-passing or insufficient-evidence clauses, then automatically re-evaluate them.
Report & Export	Filter the full compliance matrix by status and download a two-worksheet Excel review report.
Engine details	Expand details about the loaded classification model, hybrid retrieval model, and nine-family rule engine.
New project (home-page card)	Enter a custom project name, create a blank review workspace, and then upload tender, addendum, and bid PDFs.

The following environment variables customize runtime settings:

Environment variable	Default	Purpose
DEEPSEEK_API_KEY	Read from the root .env file or terminal environment	Calls the DeepSeek model for live structured slot extraction and generates intelligent remediation suggestions on the Remediation page.
TENDERCHECK_DATA_DIR	SystemCode/data/app	Storage directory for the SQLite database tendercheck.db, uploaded project files, and bid-page edit log bid_page_edits.json.

4 User Workflow and Interface Guide

The following steps use screenshots of the running interface (Figures C.2–C.7) to explain the standard tender-compliance review workflow and each button's function.

4.1 Step One: Create a Review Project (New Project)

Enter a project name, such as a contract number or lot name, in the New project card on the home page. Select + Create project to create the project and move automatically to the Overview document-management page. Existing projects can also be selected from the Project dropdown in the left sidebar.

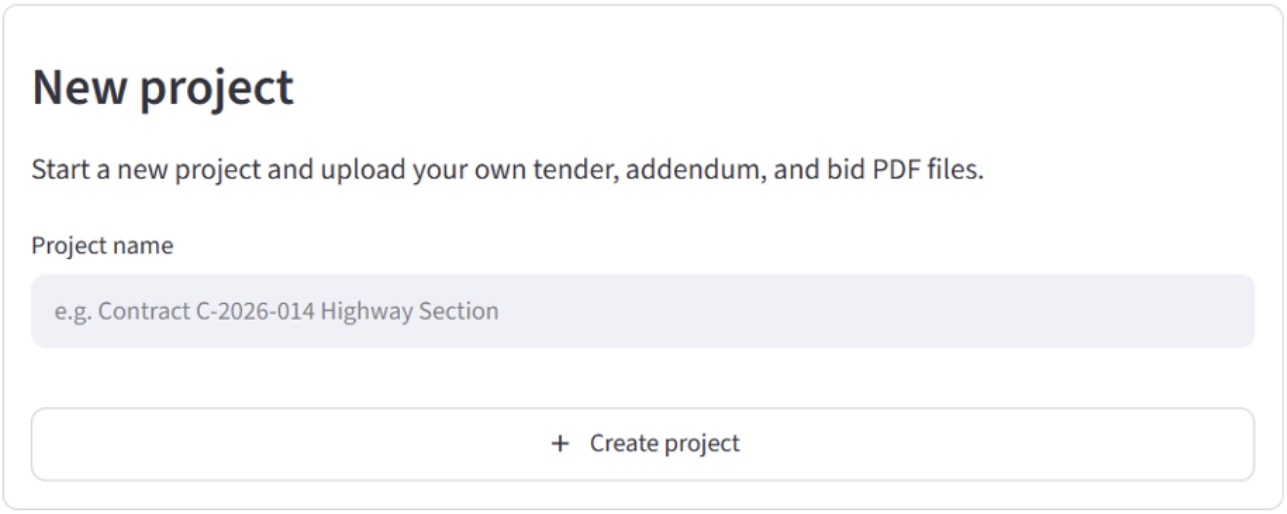


Figure C.2. New project form for creating a review project.

Input or button	Meaning and function
Project name	Enter a clear project name. It cannot be blank. The system uses it to distinguish document collections and review histories belonging to different projects.
+ Create project	Create the project record and open its Overview page, where tender, addendum, and bid PDFs can be uploaded.

4.2 Step Two: Confirm and Upload Project Documents (Overview - Step 1)

On entering Overview, first confirm the review package's completeness in Step 1 (Confirm Project Documents). The system groups files into Tender & Addenda, Primary Bid Volumes, and Supporting Attachments, and displays the page count and parsing status of each PDF.

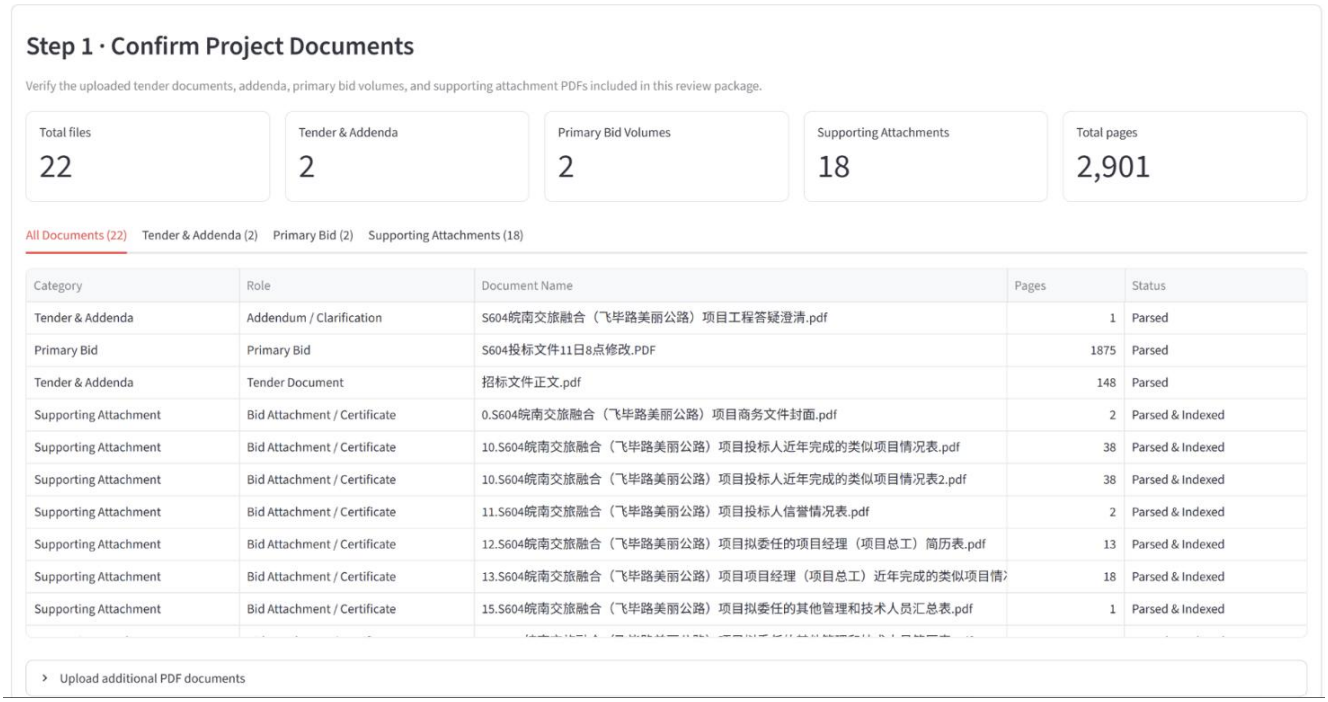


Figure C.3. Project document inventory, category totals, and additional-upload panel (Overview - Step 1).

Control or button	Meaning and function
Top statistics cards (Total files / Pages, etc.)	Show the current project's total file count, tender and addendum count, primary-bid count, supporting-attachment count, and page count so completeness can be checked quickly.
Document-category tabs (All Documents, etc.)	Switch among All Documents, Tender & Addenda, Primary Bid, and Supporting Attachments to filter the file list by role.
Document inventory (Category / Role / Status)	Show each file's role, name, page count, and parsing state. Parsed means page parsing is complete; Parsed & Indexed means the file has entered the evidence index.
Upload additional PDF documents	Expand the upload panel at the bottom to add tender files, clarification addenda, bid volumes, or qualification attachments. The system checks PDF signatures and SHA-256 hashes to avoid duplicate uploads.

4.3 Step Three: Start Automated Verification and View the Review Summary (Overview - Steps 2 and 3)

Once the file set is complete, start the automated compliance-review pipeline in Step 2. The system successively extracts and classifies tender clauses, performs multi-document hybrid RAG evidence retrieval, aligns global context with structured slots, and applies symbolic rules. After the run, Step 3 shows summary statistics for that review round.

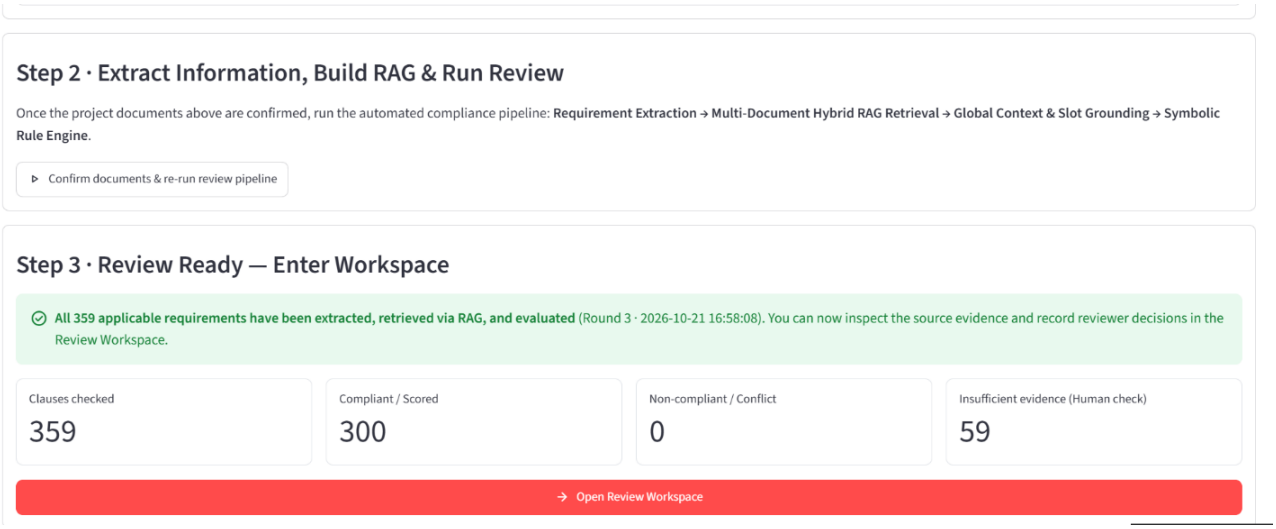


Figure C.4. Starting the automated pipeline and viewing the summary (Overview - Steps 2 and 3).

Control or button	Meaning and function
Confirm documents & re-run review pipeline	Confirm the current file set and start or rerun the full automated compliance check. The back end records live progress and creates a new Review Round.
Clauses checked / Compliant / Scored	Show the number of checkable tender requirements, items judged Compliant by the rule engine, and scoring items marked Scored.
Non-compliant / Conflict	Show non-compliant items that violate explicit limits, omit a required commitment, or contain inconsistent information (Conflicting Evidence).
Insufficient evidence (Human check)	Show items requiring focused human review, such as physical seal and signature checks, external credit-site lookups, or subjective scoring that cannot be resolved automatically.
Open Review Workspace	Open Review Workspace to inspect clause-level findings and supporting evidence.

4.4 Step Four: Filter Clauses and Compare Tender and Bid Source Pages (Review Workspace - Upper Area)

The upper area of Review Workspace provides four filter dimensions, a clause-navigation list on the left, and side-by-side original tender and bid PDF viewers on the right. Reviewers can first filter for risk items, then compare the passages highlighted in the two documents.

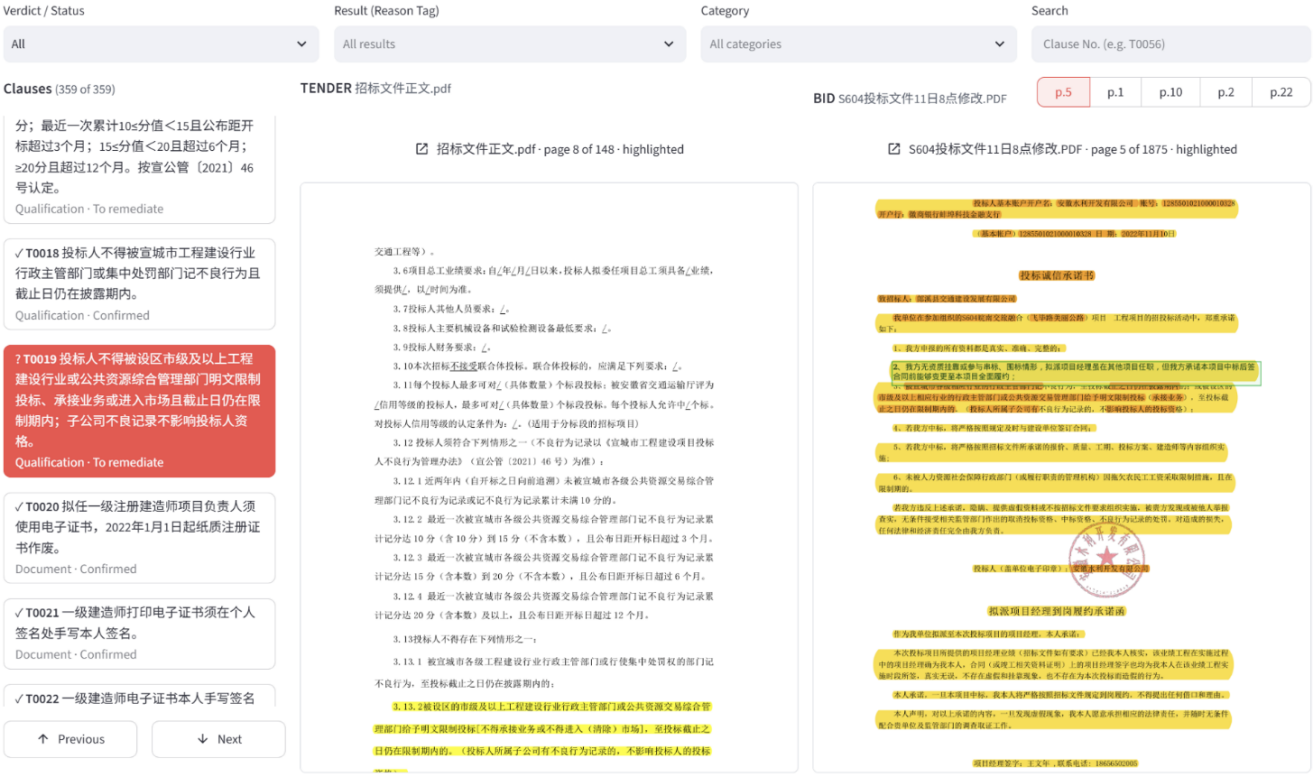


Figure C.5. Multi-dimensional filters, clause navigation, and highlighted tender/bid pages (Review Workspace - upper area).

Filter or navigation control	Meaning and function
Verdict / Status	Filter by the system verdict (Compliant, Non-compliant, or Insufficient evidence) and by human-review state (Unreviewed, Confirmed, To remediate, or Resolved).
Result (Reason Tag)	Filter by detailed result or reason, including Compliant, Scored, Non-compliant, Conflicting Evidence, Seal and Signature Check, External Web Check, Performance Linkage, Subjective Scoring, and Attachment / Formula.
Category	Filter by the eight business categories identified by the classifier: Qualification, Document, Technical, Commercial, Schedule, Consistency, Integrity, and Scoring.
Search	Enter a clause ID (such as T0019 or T0056) or a requirement keyword to locate a specific tender requirement quickly.
Clauses list / Previous / Next	Show clauses matching the filters, with ID and review status. Select a card or use Previous / Next to move through clauses.
BID page buttons (p.5 / p.1 / p.10, etc.)	Switch among bid-evidence pages returned by hybrid retrieval. The PDF viewer follows the chosen page and highlights the source passage; reviewers can also scroll the full page.

4.5 Step Five: Compare Fields, Inspect the Reasoning Chain, and Decide on Human Review (Review Workspace - Lower Area)

Scroll to the lower area of Review Workspace. The clause-comparison panel brings together the selected requirement's source text, the five highest-ranked bid passages, a structured field-comparison table, a detailed reason explanation, and reviewer action buttons. The original machine verdict and human review state are stored separately for an auditable process.



Figure C.6. Structured field comparison, reason prompt, and human-review controls (Review Workspace - lower area).

Field or button	Meaning and review function
Tender Requirement and status badges	Show the selected clause's source requirement text. Badges at the upper right separately show the machine verdict and human-review state.
Evidence tabs (E2 p.5 / E1 p.1, etc.)	Show the five highest-ranked bid-evidence passages and their page numbers. Selecting a tab synchronizes the Bid content panel and the PDF page above.
Bid content	Show the selected evidence passage's bid-volume name, page number, retrieval score, structured fields, and continuous text extracted from the page.
Field	Show each check dimension or constraint parsed from the tender clause, such as a required commitment or a numeric limit.
Bid response	Show the actual response value, original commitment text, or certificate fact extracted from the selected bid evidence and checked against the source.
Result and reason explanation	Show the rule engine's field-level finding (Compliant, Non-compliant, or Insufficient evidence), followed by the reason and suggested human check. For example, External Web Check calls for verification on an external public website.
Reasoning chain	Expand the full record of classification, evidence retrieval, slot extraction, and triggered rules for this clause.
Confirm	Record the reviewer's decision in the audit history and move automatically to the next unreviewed or non-passing clause.
To remediation	Mark the clause To remediate and open Remediation with the current clause and relevant bid page selected.

4.6 Step Six: Edit Bid Text Online and Re-evaluate Automatically (Remediation)

Selecting To remediation in the workspace, or Remediation in the left navigation, opens the bid-remediation interface shown in Figure C.7. Users can switch among open clauses on the left. On the right, they can inspect the tender requirement, request an AI editing suggestion, or directly edit the relevant bid-page text and save it to trigger re-evaluation.

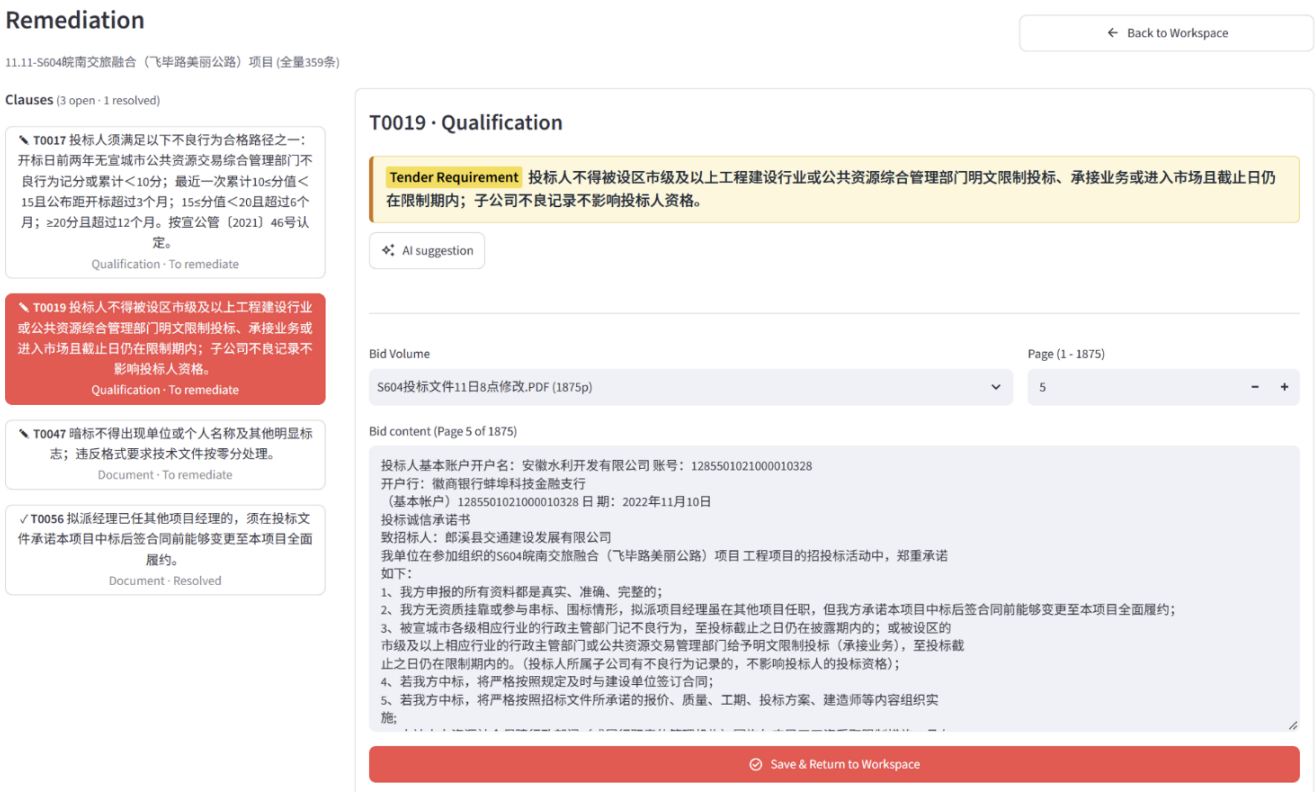


Figure C.7. Online bid-text editing, AI suggestions, and save-and-re-evaluate workflow (Remediation).

Control or button	Meaning and function
Clauses list (open / resolved)	Collect clauses in the remediation workflow. Switch between items awaiting correction (To remediate) and items that have passed after correction (Resolved).
Back to Workspace	Return directly to Review Workspace without saving edits that have not been submitted.
AI suggestion	Use the current tender requirement and bid-page text to draft a targeted compliant commitment or correction for the reviewer to consider.
Bid Volume and Page selectors (- / +)	Choose the bid volume to edit and move to the target page with the page-number controls.
Bid content editor	Edit the current bid page's text. Changes are stored as an overlay edit record, while the original text is retained for audit comparison.
Save & Return to Workspace	Save the revised bid-page text and rerun slot extraction and rule checks (R01–R09) for the clause. If the new result is Compliant, the clause closes as Resolved; the interface then returns to Review Workspace.

4.7 Step Seven: Filter the Compliance Matrix and Export the Review Report (Report & Export)

After clause review and remediation, select Report & Export from the left navigation. Summary indicators show total clauses, compliant items, non-passing items, and remediation progress. Filter the on-screen compliance matrix by All clauses, Non-passing clauses, or Remediated clauses. The export button downloads an Excel review report with a Compliance Matrix worksheet for full clause comparisons and an Issues & Remediation worksheet for issue and correction records, supporting team handover and archiving.